

Cuestionario de estudio

Java 17 & 21 — Certificación Oracle OCP

Basado en los temas del curso "Curso Certificación Java 17 & 21"

Canal Hello World Java — Miguel Rugerio

168 preguntas de opción múltiple organizadas en 24 categorías, alineadas con los objetivos del examen Oracle Certified Professional Java SE 17/21 Developer (1Z0-829).

Cómo usar esta guía:

- Responde primero todas las preguntas de una categoría sin mirar las respuestas.
- Anota tus respuestas (A, B, C o D) en una hoja aparte.
- Al final del documento encontrarás la sección de Respuestas y Explicaciones, organizada igual que las preguntas.
- Repite las categorías donde falles más de una pregunta hasta dominar el 90% o más.

Índice de categorías

1. Fundamentos JVM/JDK — 6 preguntas
2. Primitivos, casting y var — 7 preguntas
3. Operadores y control de flujo — 6 preguntas
4. Arrays y varargs — 6 preguntas
5. Strings y bloques de texto — 7 preguntas
6. POO: clases, constructores y encapsulamiento — 7 preguntas
7. Herencia y polimorfismo — 7 preguntas
8. Interfaces y clases abstractas — 7 preguntas
9. Enums — 6 preguntas
10. Records — 6 preguntas
11. Sealed classes y pattern matching — 7 preguntas
12. Generics y covarianza — 7 preguntas
13. Comparable y Comparator — 6 preguntas
14. Lambdas e interfaces funcionales — 7 preguntas
15. Method references — 5 preguntas
16. Streams API — 10 preguntas
17. Optional — 6 preguntas
18. Collections Framework — 8 preguntas
19. Concurrencia y virtual threads — 8 preguntas
20. Manejo de excepciones — 7 preguntas
21. BigDecimal y precisión numérica — 5 preguntas
22. Internacionalización (Locale, ResourceBundle) — 5 preguntas
23. Patrones de diseño (Strategy) — 4 preguntas
24. Clases anidadas e internas — 4 preguntas
25. API de fecha y hora (java.time) — 5 preguntas
26. Autoboxing, unboxing y wrapper classes — 4 preguntas
27. Módulos Java (JPMS) — 5 preguntas

Fundamentos JVM/JDK

1. ¿Cuál es la diferencia principal entre JDK, JRE y JVM?

- A) JDK incluye herramientas de desarrollo (javac, etc.) y el JRE; el JRE contiene la JVM y las librerías necesarias para ejecutar; la JVM ejecuta el bytecode
- B) JRE es para desarrollar, JDK es solo para ejecutar programas ya compilados
- C) La JVM compila el código fuente .java directamente a código máquina nativo sin bytecode intermedio
- D) JDK y JRE son sinónimos y ambos requieren instalar la JVM por separado

2. ¿Qué extensión tiene el archivo generado tras compilar un .java con javac?

- A) .class
- B) .jar
- C) .exe
- D) .bytecode

3. ¿Qué instrucción usarías para ejecutar directamente un archivo fuente Java de un solo archivo sin compilar manualmente (característica desde Java 11)?

- A) java MiArchivo.java
- B) javac -run MiArchivo.java
- C) jshell MiArchivo.java --exec
- D) java --compile MiArchivo.java

4. ¿Qué hace el Garbage Collector (recolector de basura) en Java?

- A) Libera automáticamente la memoria de objetos que ya no tienen referencias alcanzables, sin que el programador deba liberarla manualmente
- B) Compila el código fuente a bytecode
- C) Elimina variables locales no usadas antes de compilar
- D) Cierra automáticamente todos los sockets y archivos abiertos

5. ¿Qué firma debe tener obligatoriamente el método main para que la JVM pueda ejecutar un programa?

- A) public static void main(String[] args)
- B) public void main(String[] args)
- C) static void Main(String args)
- D) public static int main(String... args)

6. ¿Qué es JShell, introducido en Java 9?

- A) Una herramienta REPL (Read-Eval-Print Loop) que permite ejecutar fragmentos de código Java de forma interactiva sin crear un proyecto completo
- B) Un nuevo compilador que reemplaza a javac
- C) Un shell del sistema operativo escrito en Java
- D) Una herramienta exclusiva para depurar hilos concurrentes

Primitivos, casting y var

1. ¿Cuáles son los 8 tipos primitivos de Java?

- A) byte, short, int, long, float, double, char, boolean
- B) byte, short, int, long, float, double, String, boolean
- C) int, long, float, double, char, boolean, Integer, Double
- D) byte, int, long, double, char, boolean, var, Object

2. ¿Qué imprime este código?

int x = 130;

byte b = (byte) x;

System.out.println(b);

```
int x = 130;
byte b = (byte) x;
System.out.println(b);
```

- A) -126
- B) 130
- C) Error de compilación
- D) 0

3. ¿Cuál de las siguientes declaraciones con 'var' NO compila?

- A) var x; (sin inicializador)
- B) var lista = new ArrayList<String>();
- C) var texto = "hola";
- D) var numero = 10;

4. ¿Se puede usar 'var' para declarar un atributo (campo) de una clase?

- A) No, 'var' solo es válido para variables locales (dentro de métodos, bloques, for, try-with-resources)
- B) Sí, siempre que el campo sea private
- C) Sí, pero solo si el campo es static final
- D) Sí, en cualquier contexto incluyendo parámetros de métodos

5. ¿Qué es el 'autoboxing' en Java?

- A) La conversión automática que realiza el compilador de un tipo primitivo a su clase wrapper correspondiente (por ejemplo, int a Integer)
- B) Un proceso manual que el programador debe invocar explícitamente con cast
- C) Una técnica para envolver excepciones dentro de otras excepciones
- D) La conversión de un array a una lista

6. ¿Qué imprime este código?

double d = 9.99;

int i = (int) d;

System.out.println(i);

```
double d = 9.99;  
int i = (int) d;  
System.out.println(i);
```

- A) 9
- B) 10
- C) 9.99
- D) Error de compilación

7. ¿Cuál de las siguientes asignaciones requiere un cast explícito para compilar?

- A) `int x = (int) 3.5;` (double a int)
- B) `long l = 100;` (int a long)
- C) `double d = 100;` (int a double)
- D) `float f = 100L;` (long literal a float con widening)

Operadores y control de flujo

1. ¿Qué imprime el siguiente switch expression?

```
int dia = 3;  
String r = switch(dia){  
case 1,7 -> "Fin de semana";  
default -> "Entre semana";  
};  
System.out.println(r);
```

```
int dia = 3;  
String r = switch(dia){  
case 1, 7 -> "Fin de semana";  
default -> "Entre semana";  
};  
System.out.println(r);
```

- A) Entre semana
- B) Fin de semana
- C) Error de compilación
- D) 3

2. ¿Qué palabra clave se usa en un switch expression para devolver un valor calculado dentro de un bloque {}?

- A) yield
- B) return
- C) break
- D) value

3. ¿Cuál es el resultado de: `System.out.println(5 / 2);` y luego `System.out.println(5.0 / 2);`?

- A) 2 y 2.5
- B) 2.5 y 2.5
- C) 2 y 2
- D) 2.5 y 2

4. ¿Qué diferencia hay entre los operadores '&&' y '&' cuando se usan con dos expresiones booleanas?

- A) '&&' usa cortocircuito (no evalúa el segundo operando si el primero ya determina el resultado); '&' siempre evalúa ambos operandos
- B) '&' es solo para booleanos y '&&' solo para enteros
- C) No hay ninguna diferencia entre ambos
- D) '&&' siempre evalúa ambos operandos y '&' usa cortocircuito

5. ¿Qué imprime este bucle?

```
for(int i = 0; i < 3; i++){  
    if(i == 1) continue;  
    System.out.print(i);  
}
```

```
for(int i = 0; i < 3; i++){  
    if(i == 1) continue;  
    System.out.print(i);  
}
```

- A) 02
- B) 012
- C) 01
- D) 2

6. ¿Es obligatorio el 'break' al final de cada rama en un switch statement clásico (con dos puntos ':')?

- A) No es obligatorio sintácticamente, pero sin él el flujo 'cae' (fall-through) a la siguiente rama, lo cual suele ser un error común
- B) Sí, el compilador rechaza un switch clásico sin break en cada case
- C) El break solo es válido dentro de bucles, nunca en un switch
- D) Solo es necesario en el último case del switch

Arrays y varargs

1. ¿Cuál declaración de un método varargs es válida?

- A) void metodo(String... nombres)
- B) void metodo(String nombres...)
- C) void metodo(...String nombres)
- D) void metodo(String[]... nombres, int x)

2. ¿Qué ocurre si un método tiene un parámetro varargs y otro sobrecargado con array explícito del mismo tipo?

- A) Compila; Java prefiere la coincidencia exacta (el método con array explícito) antes que expandir varargs
- B) Error de compilación por ambigüedad siempre
- C) Siempre se ejecuta la versión varargs
- D) Solo compila si varargs se declara primero en el archivo

3. `int[] a = new int[3];` ¿cuál es el valor por defecto de cada elemento?

- A) 0
- B) null
- C) Error de compilación, hay que inicializar
- D) -1

4. ¿Qué imprime este código?

```
int[][] matriz = {{1,2},{3,4,5}};
```

```
System.out.println(matriz[1].length);
```

```
int[][] matriz = {{1,2},{3,4,5}};  
System.out.println(matriz[1].length);
```

- A) 3
- B) 2
- C) 5
- D) Error de compilación

5. ¿Qué método de la clase `Arrays` permite convertir un array en una representación de texto legible, como `"[1, 2, 3]"`?

- A) `Arrays.toString(array)`
- B) `array.print()`
- C) `Arrays.display(array)`
- D) `String.valueOf(array)` directamente muestra el contenido

6. ¿Qué ocurre al intentar acceder a un índice fuera de rango de un array, por ejemplo `array[10]` en un array de tamaño 5?

- A) Se lanza `ArrayIndexOutOfBoundsException` en tiempo de ejecución
- B) El compilador rechaza el código
- C) Devuelve null silenciosamente
- D) Devuelve 0 silenciosamente

Strings y bloques de texto

1. ¿Por qué se dice que String es inmutable en Java?

- A) Porque una vez creado un objeto String, su contenido no puede modificarse; operaciones como concat() devuelven un nuevo objeto
- B) Porque String no permite ser referenciado por más de una variable
- C) Porque el compilador prohíbe declarar más de un String por clase
- D) Porque String usa arrays mutables internamente sincronizados

2. ¿Qué imprime?

String a = "hola";

String b = "hola";

System.out.println(a == b);

```
String a = "hola";
String b = "hola";
System.out.println(a == b);
```

- A) true
- B) false
- C) Error de compilación
- D) Depende del JIT

3. ¿Cuál es la sintaxis correcta para iniciar un bloque de texto (text block, Java 15+)?

- A) Triple comillas dobles """" seguidas de salto de línea
- B) Comillas simples triples '''
- C) Corchetes angulares <"">
- D) Prefijo tb" seguido de comillas dobles

4. ¿Qué método se usa para comparar el contenido de dos Strings (no la referencia)?

- A) equals()
- B) ==
- C) compareRef()
- D) same()

5. ¿Por qué se recomienda StringBuilder en lugar de concatenar Strings con '+' dentro de un bucle con muchas iteraciones?

- A) Porque cada concatenación con '+' crea un nuevo objeto String (dado que String es inmutable), lo que es ineficiente; StringBuilder modifica un buffer mutable interno
- B) Porque el operador '+' no está permitido con Strings en bucles
- C) Porque StringBuilder es la única clase que soporta texto en Java
- D) Porque '+' solo funciona fuera de bucles

6. ¿Qué imprime?

String s = " Hola Mundo ";

System.out.println(s.strip().length());

```
String s = " Hola Mundo ";  
System.out.println(s.strip().length());
```

A) 10

B) 14

C) 9

D) 11

7. ¿Qué método de String permite dividir el texto en un array usando un delimitador, por ejemplo una coma?

A) split(regex)

B) divide(regex)

C) tokenize(regex)

D) partition(regex)

POO: clases, constructores y encapsulamiento

1. ¿Qué ocurre si una clase no define ningún constructor explícito?

- A) El compilador genera automáticamente un constructor por defecto sin argumentos
- B) La clase no puede ser instanciada nunca
- C) Java lanza una excepción en tiempo de ejecución al usar new
- D) Se genera un constructor con todos los atributos como parámetros

2. ¿Qué significa que un atributo sea 'static'?

- A) Pertenece a la clase y es compartido por todas las instancias, en lugar de pertenecer a cada objeto individual
- B) No puede modificarse nunca después de asignarse
- C) Solo puede accederse desde el método main
- D) Se destruye automáticamente al finalizar cada método

3. ¿Cuál es el propósito principal del modificador 'final' aplicado a una variable?

- A) Impedir que la variable sea reasignada después de su primera asignación
- B) Hacer que la variable sea accesible desde cualquier paquete
- C) Convertir la variable en static automáticamente
- D) Permitir que la variable se sobrescriba en subclases

4. ¿Puede una clase declarada como 'final' ser extendida (heredada)?

- A) No, una clase final no puede tener subclases
- B) Sí, siempre
- C) Solo si está en el mismo paquete
- D) Solo si implementa una interfaz

5. ¿Qué es un bloque de inicialización estático (static initializer block) y cuándo se ejecuta?

- A) Un bloque static { ... } que se ejecuta una única vez, cuando la clase se carga por primera vez en la JVM, antes de crear cualquier instancia
- B) Un bloque que se ejecuta cada vez que se crea una nueva instancia de la clase
- C) Un bloque que solo puede contener llamadas a métodos privados
- D) Un bloque que reemplaza al constructor por completo

6. ¿Cuál es el propósito principal del encapsulamiento en POO?

- A) Ocultar el estado interno de un objeto y exponer solo una interfaz pública controlada (getters/setters), protegiendo la integridad de los datos
- B) Permitir que todas las clases accedan directamente a los atributos de cualquier otra clase
- C) Eliminar la necesidad de constructores
- D) Convertir automáticamente todos los métodos en static

7. ¿Qué hace la palabra clave 'this' dentro de un constructor cuando se usa como this(...)?

- A) Invoca a otro constructor sobrecargado de la misma clase (constructor chaining), y debe ser la primera línea del constructor
- B) Invoca al constructor de la superclase
- C) Crea una nueva instancia adicional de la clase actual
- D) Es equivalente a static en ese contexto

Herencia y polimorfismo

1. ¿Qué es 'overriding' (sobrescritura) de métodos?

- A) Redefinir en una subclase un método con la misma firma que el de la superclase, cambiando su implementación
- B) Definir varios métodos con el mismo nombre pero distintos parámetros en la misma clase
- C) Declarar un método como private en una subclase
- D) Llamar a un método estático desde una instancia

2. ¿Qué palabra clave permite invocar el constructor o método de la superclase desde una subclase?

- A) super
- B) this
- C) parent
- D) extends

3. Dado:

```
class Animal { void sonido(){ System.out.println("..."); } }
```

```
class Perro extends Animal { void sonido(){ System.out.println("Guau"); } }
```

```
Animal a = new Perro();
```

```
a.sonido();
```

¿Qué se imprime?

```
class Animal { void sonido(){ System.out.println("..."); } }  
class Perro extends Animal { void sonido(){ System.out.println("Guau"); } }  
Animal a = new Perro();  
a.sonido();
```

- A) Guau
- B) ...
- C) Error de compilación
- D) Guau seguido de ...

4. Java permite herencia múltiple de clases pero sí de interfaces. ¿Verdadero o falso?

- A) Falso: Java NO permite herencia múltiple de clases (solo una superclase), pero SÍ permite implementar múltiples interfaces
- B) Verdadero, ambas cosas son posibles sin restricción
- C) Falso, tampoco permite implementar múltiples interfaces
- D) Verdadero, pero solo desde Java 21

5. ¿Qué modificador de acceso permite que un miembro sea visible solo dentro del mismo paquete y en subclases de otros paquetes?

- A) protected
- B) private
- C) default (sin modificador)
- D) public

6. ¿Puede un método sobrescrito (override) en una subclase declarar una excepción checked más amplia que la del método original de la superclase?

- A) No, el método sobrescrito solo puede declarar la misma excepción checked, una subclase de ella, o ninguna; nunca una más amplia
- B) Sí, siempre puede declarar cualquier excepción checked adicional
- C) Solo si el método es static
- D) Solo si la superclase es una interfaz

7. ¿Qué es el 'binding estático' (static binding) frente al 'binding dinámico' (dynamic binding)?

- A) El binding estático resuelve en compilación qué método se llama (métodos static, private, final, y sobrecarga); el binding dinámico resuelve en tiempo de ejecución (métodos sobrescritos vía polimorfismo)
- B) Ambos se resuelven siempre en tiempo de compilación
- C) El binding dinámico solo aplica a atributos, nunca a métodos
- D) El binding estático solo existe en interfaces

Interfaces y clases abstractas

1. ¿Qué permiten los métodos 'default' en una interfaz (desde Java 8)?

- A) Proveer una implementación concreta dentro de la interfaz que las clases implementadoras pueden usar o sobrescribir
- B) Hacer que el método sea abstracto obligatoriamente
- C) Convertir la interfaz en una clase abstracta automáticamente
- D) Ejecutarse solo si la clase implementadora es static

2. ¿Puede una clase abstracta tener constructores?

- A) Sí, aunque no se pueda instanciar directamente, sus constructores se ejecutan al instanciar subclases concretas
- B) No, nunca
- C) Solo si es final
- D) Solo si no tiene métodos abstractos

3. ¿Qué es una clase anónima en Java?

- A) Una clase sin nombre que se define e instancia en un único paso, útil para implementar una interfaz o extender una clase de forma puntual
- B) Una clase que no puede tener métodos
- C) Un tipo de enum sin constantes
- D) Una clase que solo puede usarse dentro de records

4. ¿Cuántos métodos abstractos puede tener como máximo una interfaz funcional?

- A) Exactamente uno (además de métodos default/static)
- B) Cero
- C) Máximo dos
- D) Ilimitados

5. ¿Puede una interfaz declarar campos (variables)?

- A) Sí, pero son implícitamente public static final (constantes), no se pueden tener campos de instancia mutables
- B) No, las interfaces nunca pueden declarar variables
- C) Sí, y son mutables como en una clase normal
- D) Solo si la interfaz es sealed

6. ¿Qué son los métodos 'static' dentro de una interfaz (desde Java 8)?

- A) Métodos con implementación que pertenecen a la interfaz misma y se invocan como NombreInterfaz.metodo(), no se heredan a las clases implementadoras
- B) Métodos que deben ser sobrescritos obligatoriamente por cada clase implementadora
- C) Métodos abstractos adicionales
- D) Un sinónimo de los métodos default

7. Si una clase implementa dos interfaces que definen el mismo método default con distinta implementación, ¿qué ocurre?

- A) La clase debe sobrescribir explícitamente ese método para resolver la ambigüedad, o el código no compila
- B) Java elige automáticamente la implementación de la primera interfaz declarada
- C) Se produce un error solo en tiempo de ejecución, no en compilación
- D) Ambas implementaciones se ejecutan en secuencia automáticamente

Enums

1. ¿Pueden los enums en Java tener constructores, campos y métodos?

- A) Sí, un enum es una clase especial que puede tener constructores (siempre implícitamente privados), campos y métodos, incluso implementar interfaces
- B) No, un enum solo puede contener constantes simples
- C) Solo puede tener métodos static
- D) Solo puede implementar una interfaz si no tiene constructor

2. ¿Qué método permite obtener la posición ordinal (basada en 0) de una constante enum?

- A) ordinal()
- B) position()
- C) index()
- D) value()

3. ¿Se puede usar un enum en un switch statement/expression sin prefijo del nombre de la clase en los 'case'?

- A) Sí, solo se usa el nombre de la constante (ej. case LUNES ->)
- B) No, siempre hay que escribir DiaSemana.LUNES
- C) Solo si el enum es public
- D) Solo en switch clásico, no en switch expression

4. ¿Qué método permite obtener todas las constantes de un enum como un array?

- A) NombreEnum.values()
- B) NombreEnum.getAll()
- C) NombreEnum.list()
- D) NombreEnum.entries()

5. ¿Puede un enum implementar un cuerpo de clase distinto para cada constante (constant-specific class body)?

- A) Sí, cada constante puede sobrescribir un método abstracto declarado en el enum con su propia implementación
- B) No, todas las constantes deben compartir exactamente el mismo comportamiento
- C) Solo si el enum no tiene constructor
- D) Solo es posible con dos constantes como máximo

6. ¿Qué excepción se lanza si se llama a valueOf() de un enum con un nombre que no corresponde a ninguna constante?

- A) IllegalArgumentException
- B) NullPointerException
- C) ClassCastException
- D) NoSuchFieldException

Records

1. ¿Qué genera automáticamente el compilador al declarar un record?

- A) Constructor canónico, métodos de acceso (accessors) con el nombre del componente, equals(), hashCode() y toString()
- B) Solo el método toString()
- C) Getters con prefijo 'get' como en un JavaBean tradicional
- D) Setters para modificar cada componente

2. ¿Son mutables los componentes de un record?

- A) No, todos los campos de un record son implícitamente final e inmutables
- B) Sí, se pueden reasignar con setters generados
- C) Solo si se declaran como var
- D) Depende de si el record implementa Serializable

3. ¿Qué es un 'constructor compacto' en un record?

- A) Una forma de escribir el constructor canónico sin repetir la lista de parámetros, útil para validar o normalizar valores antes de la asignación implícita
- B) Un constructor que no acepta parámetros
- C) Un constructor privado obligatorio en todo record
- D) Un método estático que reemplaza al constructor

4. ¿Puede un record implementar interfaces?

- A) Sí, un record puede implementar una o varias interfaces, igual que una clase normal
- B) No, los records no pueden implementar ninguna interfaz
- C) Solo pueden implementar Comparable
- D) Solo pueden implementar Serializable

5. ¿Puede un record declarar campos de instancia adicionales fuera de los componentes definidos en su cabecera?

- A) No, un record solo puede tener los campos correspondientes a sus componentes declarados; sí puede tener campos static adicionales
- B) Sí, puede declarar cualquier campo de instancia adicional libremente
- C) Sí, pero solo si son públicos
- D) Sí, pero solo si el record es sealed

6. ¿Qué genera el compilador si defines explícitamente el accessor de un componente de un record, por ejemplo `public int x() { return x * -1; }`?

- A) Usa la implementación personalizada en lugar del accessor generado automáticamente, permitiendo modificar qué se devuelve
- B) Produce un error de compilación porque no se puede sobrescribir un accessor
- C) Ignora la implementación personalizada y usa siempre la generada
- D) Solo funciona si el método se declara private

Sealed classes y pattern matching

1. ¿Para qué sirve la palabra clave 'sealed' en una clase o interfaz (Java 17)?

- A) Para restringir explícitamente qué clases pueden extenderla o implementarla, usando la cláusula 'permits'
- B) Para hacerla immutable automáticamente como un record
- C) Para prohibir que tenga métodos abstractos
- D) Para convertirla automáticamente en final

2. Una subclase de una clase sealed, ¿qué modificador debe declarar obligatoriamente?

- A) final, sealed o non-sealed
- B) static
- C) abstract siempre
- D) private

3. ¿Qué permite el pattern matching for instanceof (Java 16+)?

- A) Combinar la comprobación instanceof con la asignación a una variable en una sola expresión, evitando el cast manual: `if (obj instanceof String s) {...}`
- B) Comparar dos instancias por referencia
- C) Convertir automáticamente cualquier objeto a String
- D) Verificar solo si un objeto es null

4. En Java 21, ¿qué aporta el pattern matching para switch sobre records (record patterns)?

- A) Permite desestructurar los componentes de un record directamente en el 'case' (ej. `case Punto(int x, int y) -> ...`)
- B) Permite modificar los valores del record dentro del switch
- C) Obliga a que el record implemente Comparable
- D) Solo funciona con enums, no con records

5. ¿Pueden las clases permitidas ('permits') de una clase sealed estar en un paquete diferente?

- A) Sí, pero solo si pertenecen al mismo módulo, o si están en el mismo paquete cuando no se usan módulos
- B) No, siempre deben estar exactamente en el mismo archivo .java
- C) Sí, sin ninguna restricción de paquete o módulo
- D) Solo si son records

6. ¿Qué ventaja aporta un switch exhaustivo sobre una jerarquía sealed junto con pattern matching?

- A) El compilador puede verificar que se cubren todos los subtipos posibles, sin necesitar una rama 'default', mejorando la seguridad en compilación
- B) Permite que el switch se ejecute en paralelo automáticamente
- C) Elimina la necesidad de declarar 'permits'
- D) Convierte automáticamente el sealed en un enum

7. ¿Qué patrón usarías en un case para descomponer un record Punto(int x, int y) y aplicar una guarda adicional 'cuando x sea positivo' (Java 21, guarded patterns)?

- A) case Punto(int x, int y) when x > 0 -> ...
- B) case Punto(int x, int y) if x > 0 -> ...
- C) case Punto(int x, int y) && x > 0 -> ...
- D) No es posible añadir condiciones adicionales dentro de un case

Generics y covarianza

1. ¿Qué problema resuelven los Generics en Java?

- A) Proveen seguridad de tipos en tiempo de compilación para colecciones y clases, evitando ClassCastException en tiempo de ejecución y castings manuales
- B) Permiten que una variable cambie de tipo dinámicamente en tiempo de ejecución
- C) Eliminan la necesidad de interfaces
- D) Sirven solo para arrays primitivos

2. ¿Qué significa el comodín '?' extends Number' en una declaración de generics?

- A) Acepta el tipo Number o cualquiera de sus subtipos (útil para lectura/producción de datos, 'Producer Extends')
- B) Acepta solo el tipo Number exacto
- C) Acepta cualquier tipo excepto Number
- D) Permite modificar la colección añadiendo elementos de cualquier subtipo libremente

3. ¿Por qué List<String> NO es un subtipo de List<Object> en Java (a diferencia de los arrays)?

- A) Porque los generics son invariantes: aunque String sea subtipo de Object, List<String> y List<Object> son tipos distintos sin relación de subtipo entre sí
- B) Porque List<String> no puede contener objetos de tipo Object
- C) Porque los generics no soportan herencia en absoluto
- D) Porque solo los arrays son válidos en Java, List<> es obsoleto

4. ¿Qué es 'type erasure' (borrado de tipos) en Java?

- A) El proceso por el cual la información de tipo genérico se elimina en tiempo de compilación, quedando solo el tipo raw (u Object) en el bytecode
- B) Un proceso que elimina variables no usadas del código fuente
- C) Una técnica para borrar clases genéricas del classpath
- D) Un mecanismo del recolector de basura para objetos genéricos

5. ¿Qué significa el comodín '?' super Integer' en una declaración de generics?

- A) Acepta Integer o cualquiera de sus supertipos (útil para escribir/consumir datos, 'Consumer Super')
- B) Acepta solo subtipos de Integer
- C) Acepta cualquier tipo excepto Integer
- D) Es sintácticamente inválido en Java

6. ¿Se puede crear un array de un tipo genérico parametrizado, como new List<String>[10]?

- A) No, Java no permite crear directamente arrays de tipos genéricos parametrizados debido al conflicto entre la covarianza de arrays y el borrado de tipos
- B) Sí, sin ninguna restricción
- C) Solo si el tipo genérico es Object
- D) Solo dentro de métodos static

7. ¿Qué es un método genérico y cómo se declara su parámetro de tipo?

- A) Un método que declara su propio parámetro de tipo antes del tipo de retorno, por ejemplo: `public <T> T primero(List<T> lista)`
- B) Un método que solo puede usarse dentro de clases genéricas
- C) Un método que acepta cualquier objeto sin necesidad de declarar tipo alguno
- D) Un método que siempre debe devolver `Object`

Comparable y Comparator

1. ¿Cuál es la diferencia principal entre Comparable y Comparator?

- A) Comparable se implementa en la propia clase para definir su orden natural (compareTo); Comparator se define externamente y permite múltiples estrategias de ordenación (compare)
- B) Comparable es para colecciones y Comparator solo para arrays
- C) Comparator solo funciona con tipos primitivos
- D) No hay diferencia, son sinónimos desde Java 17

2. ¿Qué valor devuelve compareTo() si el objeto actual es 'mayor' que el que se compara?

- A) Un valor positivo
- B) Un valor negativo
- C) Siempre 1 exactamente
- D) 0

3. ¿Cómo se encadenan múltiples criterios de comparación usando Comparator (por ejemplo, por apellido y luego por nombre)?

- A) Comparator.comparing(Persona::getApellido).thenComparing(Persona::getNombre)
- B) Comparator.and(Persona::getApellido, Persona::getNombre)
- C) Comparator.merge(apellido, nombre)
- D) No es posible encadenar comparadores en Java

4. ¿Qué ocurre si dos objetos son 'iguales' según equals() pero compareTo() no devuelve 0 para ellos?

- A) Se dice que la clase tiene un orden 'inconsistente con equals', lo cual puede causar comportamientos inesperados en colecciones ordenadas como TreeSet o TreeMap
- B) Java lanza una excepción automáticamente en tiempo de compilación
- C) No tiene ningún efecto práctico en ninguna colección
- D) Se corrige automáticamente al usar Comparator en vez de Comparable

5. ¿Cómo se invierte el orden producido por un Comparator existente, por ejemplo para ordenar de mayor a menor?

- A) comparator.reversed()
- B) comparator.invert()
- C) Comparator.negate(comparator)
- D) comparator.opposite()

6. ¿Qué método estático de Comparator crea un comparador a partir de una función que extrae una clave comparable de cada elemento?

- A) Comparator.comparing(funcionExtractora)
- B) Comparator.by(funcionExtractora)
- C) Comparator.key(funcionExtractora)
- D) Comparator.extract(funcionExtractora)

Lambdas e interfaces funcionales

1. ¿Cuál es la sintaxis correcta de una expresión lambda que recibe dos enteros y devuelve su suma?

- A) (a, b) -> a + b
- B) function(a, b) => a + b
- C) lambda(a, b) { return a + b; }
- D) (a, b) => { a + b }

2. ¿Qué interfaz funcional del paquete java.util.function representa una operación que recibe un argumento y devuelve un boolean?

- A) Predicate<T>
- B) Function<T,R>
- C) Supplier<T>
- D) Consumer<T>

3. ¿Qué interfaz funcional representa una operación que NO recibe argumentos pero produce un resultado?

- A) Supplier<T>
- B) Consumer<T>
- C) Function<T,R>
- D) BiFunction<T,U,R>

4. ¿Puede una expresión lambda acceder y modificar una variable local externa capturada?

- A) No, solo puede leer variables locales externas que sean 'effectively final' (efectivamente finales); no puede reasignarlas
- B) Sí, puede modificar libremente cualquier variable local externa
- C) Solo si la variable es static
- D) Solo dentro de un Comparator

5. ¿Qué interfaz funcional recibe un argumento y no devuelve ningún resultado (void)?

- A) Consumer<T>
- B) Supplier<T>
- C) Function<T,R>
- D) Predicate<T>

6. ¿Qué interfaz funcional recibe un argumento de tipo T y produce un resultado de tipo R distinto?

- A) Function<T,R>
- B) UnaryOperator<T>
- C) Predicate<T>
- D) BiConsumer<T,U>

7. ¿Puede una expresión lambda lanzar una excepción checked si la interfaz funcional que implementa no la declara en su método abstracto?

- A) No, si el método abstracto de la interfaz funcional no declara esa excepción checked, la lambda no puede lanzarla (no compila)
- B) Sí, cualquier lambda puede lanzar cualquier excepción sin restricciones
- C) Solo si la lambda se asigna a una variable Object
- D) Solo si la excepción es unchecked, nunca aplica a checked de todos modos

Method references

1. ¿Cuál de estas es la sintaxis de una referencia a método estático?

- A) Clase::metodoEstatico
- B) Clase->metodoEstatico
- C) Clase.metodoEstatico()
- D) new Clase::metodo

2. ¿Qué representa la referencia a método `String::toUpperCase` usada como `Function<String,String>`?

- A) Una referencia a un método de instancia sobre un tipo arbitrario, equivalente a `s -> s.toUpperCase()`
- B) Una referencia a un constructor
- C) Una referencia a un método estático de la clase `String`
- D) Un error de compilación porque `toUpperCase` no es estático

3. ¿Cómo se escribe una referencia a constructor para crear instancias de `ArrayList` mediante un `Supplier`?

- A) `ArrayList::new`
- B) `new ArrayList()::create`
- C) `ArrayList.new()`
- D) `Supplier.of(ArrayList)`

4. ¿Cuál es la sintaxis de una referencia a un método de instancia sobre un objeto particular ya existente, por ejemplo `'obj::metodo'`?

- A) `objetoExistente::metodoDeInstancia` — el objeto ya está fijado como receptor de la llamada
- B) `Clase::metodoDeInstancia` sin objeto
- C) `objetoExistente->metodoDeInstancia`
- D) `new objetoExistente::metodo`

5. ¿A qué expresión lambda equivale la referencia de método `Math::max` usada como `BinaryOperator<Integer>`?

- A) `(a, b) -> Math.max(a, b)`
- B) `(a) -> Math.max(a)`
- C) `() -> Math.max()`
- D) `(a, b) -> a.max(b)`

Streams API

1. ¿Qué caracteriza a los streams en Java respecto a las colecciones que los originan?

- A) Un stream no almacena datos propios; procesa datos de una fuente de forma perezosa (lazy) y puede consumirse una sola vez
- B) Un stream modifica siempre la colección original
- C) Un stream se puede recorrer múltiples veces como una lista
- D) Un stream requiere que la colección esté ordenada previamente

2. ¿Cuál de las siguientes es una operación TERMINAL de Stream (no intermedia)?

- A) collect()
- B) filter()
- C) map()
- D) sorted()

3. ¿Qué hace el siguiente código?

List<String> nombres = List.of("Ana","Luis","Eva");

nombres.stream().filter(n -> n.length()>3).forEach(Svstem.out::println);

```
List<String> nombres = List.of("Ana", "Luis", "Eva");
nombres.stream()
    .filter(n -> n.length() > 3)
    .forEach(System.out::println);
```

- A) Imprime solo "Luis" porque es el único nombre con más de 3 caracteres
- B) Imprime los tres nombres
- C) No compila
- D) Imprime "Ana" y "Eva"

4. ¿Qué clase se usa típicamente para acumular resultados de un Stream en una colección mediante collect(), como Collectors.toList()?

- A) java.util.stream.Collectors
- B) java.util.Collection
- C) java.util.Accumulator
- D) java.util.stream.Reducer

5. ¿Qué hace Collectors.groupingBy()?

- A) Agrupa los elementos de un stream en un Map según una función clasificadora, devolviendo un Map<clave, List<elementos>>
- B) Ordena los elementos alfabéticamente
- C) Elimina duplicados del stream
- D) Convierte el stream en un array

6. ¿Cuál es la diferencia entre map() y flatMap() en Streams?

- A) map() transforma cada elemento en otro elemento uno a uno; flatMap() transforma cada elemento en un stream y luego 'aplana' todos esos streams en uno solo
- B) map() solo funciona con enteros y flatMap solo con Strings
- C) flatMap() es simplemente un alias de map()
- D) map() es una operación terminal y flatMap() es intermedia

7. ¿Qué devuelve la operación terminal reduce() en un Stream?

- A) Combina los elementos del stream en un único resultado acumulado, aplicando repetidamente una función de combinación binaria
- B) Elimina elementos duplicados
- C) Divide el stream en dos partes
- D) Ordena los elementos de forma descendente

8. ¿Cuál es la diferencia entre un stream secuencial y uno paralelo (parallelStream())?

- A) El stream paralelo divide el procesamiento entre múltiples hilos usando el ForkJoinPool común, mientras que el secuencial procesa los elementos uno a uno en un solo hilo
- B) El stream paralelo siempre garantiza mantener el orden original de los elementos en cualquier operación
- C) No hay ninguna diferencia práctica de rendimiento
- D) El stream paralelo solo puede usarse con listas de tipo primitivo

9. ¿Qué método de Stream permite limitar la cantidad de elementos procesados, por ejemplo a los primeros 5?

- A) limit(5)
- B) take(5)
- C) first(5)
- D) cut(5)

10. ¿Qué método permite obtener un IntStream de números consecutivos, por ejemplo del 1 al 10?

- A) IntStream.rangeClosed(1, 10)
- B) IntStream.of(1, 10)
- C) IntStream.sequence(1, 10)
- D) IntStream.between(1, 10)

Optional

1. ¿Cuál es el propósito principal de la clase `Optional<T>`?

- A) Representar explícitamente la posible ausencia de un valor, evitando `NullPointerException` y forzando un manejo explícito del caso 'vacío'
- B) Reemplazar por completo el uso de `null` en cualquier contexto, incluidos campos de clase
- C) Mejorar el rendimiento de las colecciones
- D) Servir como una nueva colección ordenada

2. ¿Qué hace `Optional.ofNullable(valor)` si 'valor' es `null`?

- A) Devuelve un `Optional` vacío (`Optional.empty()`) sin lanzar excepción
- B) Lanza `NullPointerException` inmediatamente
- C) Devuelve `null`
- D) No compila

3. ¿Cuál es una forma recomendada de obtener un valor por defecto de un `Optional` si está vacío?

- A) `optional.orElse(valorPorDefecto)`
- B) `optional.getDefault()`
- C) `optional.value()`
- D) `optional.orNull()`

4. ¿Qué hace `Optional.map(funcion)` si el `Optional` está vacío?

- A) Devuelve directamente un `Optional` vacío, sin aplicar la función
- B) Lanza `NoSuchElementException`
- C) Ejecuta la función igualmente con un valor `null`
- D) No compila

5. ¿Qué excepción lanza `optional.get()` si el `Optional` está vacío?

- A) `NoSuchElementException`
- B) `NullPointerException`
- C) `IllegalStateException`
- D) `OptionalNotPresentException`

6. ¿Cuál es una buena práctica respecto al uso de `Optional` como tipo de un campo de clase o parámetro de método?

- A) Se recomienda evitarlo: `Optional` está pensado principalmente como tipo de retorno de métodos, no para campos ni parámetros
- B) Se recomienda usar `Optional` en todos los campos de todas las clases
- C) `Optional` debe usarse siempre en lugar de tipos primitivos
- D) No existe ninguna recomendación al respecto

Collections Framework

1. ¿Cuál es la diferencia principal entre List, Set y Map en el Collections Framework?

- A) List permite duplicados y mantiene orden de inserción; Set no permite duplicados; Map almacena pares clave-valor
- B) List no permite duplicados; Set sí los permite; Map es solo para arrays
- C) Los tres son idénticos, solo cambia el nombre
- D) Set mantiene siempre orden alfabético automáticamente y List nunca permite acceso por índice

2. ¿Qué implementación de List garantiza acceso rápido $O(1)$ por índice, pero inserciones/eliminaciones costosas en el medio?

- A) ArrayList
- B) LinkedList
- C) Vector solamente
- D) TreeSet

3. ¿Qué característica distingue a un TreeMap de un HashMap?

- A) TreeMap mantiene sus claves ordenadas (orden natural o Comparator); HashMap no garantiza ningún orden
- B) TreeMap permite claves duplicadas y HashMap no
- C) HashMap es más lento que TreeMap en todos los casos
- D) TreeMap no permite valores null nunca y HashMap sí siempre

4. ¿Qué devuelve List.of(1,2,3) en cuanto a mutabilidad?

- A) Una lista inmutable; intentar añadir o eliminar elementos lanza UnsupportedOperationException
- B) Una lista completamente mutable equivalente a ArrayList
- C) Un array primitivo int[]
- D) Un Set en lugar de una List

5. ¿Qué debe sobrescribir correctamente una clase para usarse como clave en un HashMap de forma consistente?

- A) equals() y hashCode() (contrato de Object)
- B) Comparable obligatoriamente
- C) Serializable obligatoriamente
- D) Cloneable obligatoriamente

6. ¿Qué estructura de datos implementa típicamente una Queue de tipo FIFO (primero en entrar, primero en salir)?

- A) LinkedList (implementando Queue)
- B) TreeSet
- C) Stack únicamente
- D) HashMap

7. ¿Qué diferencia hay entre Iterator y ListIterator?

- A) ListIterator extiende Iterator añadiendo la capacidad de recorrer en ambas direcciones y modificar elementos durante la iteración (add, set)
- B) Iterator solo funciona con Map, ListIterator solo con List
- C) No hay ninguna diferencia real entre ambos
- D) ListIterator no permite eliminar elementos mientras que Iterator sí

8. ¿Qué excepción se lanza al modificar una colección (por ejemplo, con remove()) directamente mientras se itera sobre ella con un for-each, sin usar el propio Iterator?

- A) ConcurrentModificationException
- B) UnsupportedOperationException
- C) IllegalStateException
- D) NoSuchElementException

Concurrencia y virtual threads

1. ¿Qué interfaz representa una tarea que puede devolver un resultado y lanzar excepciones checked, a diferencia de Runnable?

- A) Callable<V>
- B) Runnable
- C) Thread
- D) Executor

2. ¿Qué provee un ExecutorService respecto a crear Thread manualmente?

- A) Un framework de alto nivel para administrar un pool de hilos reutilizables, enviar tareas (submit/execute) y gestionar su ciclo de vida
- B) Solo permite ejecutar un hilo a la vez
- C) Reemplaza completamente la necesidad de sincronización
- D) Es exclusivo para aplicaciones de red

3. ¿Qué son los 'virtual threads' introducidos en Java 21 (JEP 444)?

- A) Hilos ligeros gestionados por la JVM (no por el sistema operativo) que permiten crear millones de hilos concurrentes con bajo costo de memoria, ideales para aplicaciones con mucha I/O
- B) Hilos que solo pueden ejecutar código funcional (lambdas)
- C) Un reemplazo para las clases enum
- D) Hilos que se ejecutan exclusivamente en la GPU

4. ¿Qué palabra clave permite garantizar que solo un hilo a la vez ejecute un bloque de código crítico?

- A) synchronized
- B) volatile
- C) static
- D) transient

5. ¿Cómo se crea rápidamente un ExecutorService basado en virtual threads en Java 21?

- A) Executors.newVirtualThreadPerTaskExecutor()
- B) Executors.newFixedThreadPool(1)
- C) Thread.ofVirtual().createPool()
- D) new VirtualExecutorService()

6. ¿Qué método de la clase Thread permite pausar la ejecución del hilo actual durante un tiempo determinado?

- A) Thread.sleep(milisegundos)
- B) Thread.pause(milisegundos)
- C) Thread.wait(milisegundos) sin sincronización
- D) Thread.delay(milisegundos)

7. ¿Qué es una 'race condition' (condición de carrera) en programación concurrente?

- A) Una situación en la que el resultado del programa depende del orden impredecible de ejecución de múltiples hilos accediendo a datos compartidos sin la sincronización adecuada
- B) Un mecanismo que garantiza que los hilos se ejecuten siempre en el mismo orden
- C) Un tipo de excepción lanzada exclusivamente por `ExecutorService`
- D) Una técnica de optimización de rendimiento en la JVM

8. ¿Qué aporta la clase `java.util.concurrent.atomic.AtomicInteger` frente a un `int` normal en un entorno multihilo?

- A) Provee operaciones atómicas (como `incrementAndGet()`) que se ejecutan de forma indivisible sin necesidad de bloques `synchronized`, evitando condiciones de carrera
- B) Convierte automáticamente el entero en un tipo `BigInteger`
- C) Solo puede usarse fuera de hilos concurrentes
- D) Reemplaza completamente a `ExecutorService`

Manejo de excepciones

1. ¿Cuál es la diferencia entre una excepción 'checked' y una 'unchecked'?

- A) Las checked deben declararse (throws) o manejarse obligatoriamente en compilación (extienden Exception); las unchecked no obligan a manejo explícito (extienden RuntimeException)
- B) Las unchecked solo pueden lanzarse dentro de un try-catch
- C) Las checked no pueden ser capturadas nunca con catch
- D) No hay diferencia real, es solo una convención de nombres

2. ¿Qué garantiza el bloque 'finally' en un try-catch-finally?

- A) Se ejecuta siempre, haya o no excepción, incluso si hay un return dentro del try o catch (salvo System.exit() o error grave de JVM)
- B) Solo se ejecuta si no ocurrió ninguna excepción
- C) Solo se ejecuta si hubo una excepción no capturada
- D) Se ejecuta antes que el bloque try

3. ¿Qué ventaja ofrece 'try-with-resources' introducido en Java 7?

- A) Cierra automáticamente los recursos que implementan AutoCloseable al finalizar el bloque try, sin necesidad de un finally explícito
- B) Permite capturar cualquier tipo de excepción sin declarar catch
- C) Elimina la necesidad de manejar excepciones por completo
- D) Solo funciona con archivos, no con conexiones de red o BD

4. ¿Qué clase es la superclase común de Exception y Error?

- A) Throwable
- B) RuntimeException
- C) Object
- D) IOException

5. ¿Se puede tener múltiples bloques catch para un mismo try, capturando distintos tipos de excepción?

- A) Sí, se pueden encadenar varios catch; el orden importa: las excepciones más específicas deben ir antes que las más generales
- B) No, solo se permite un único catch por try
- C) Sí, pero el orden de los catch nunca importa
- D) Solo se permite si todas las excepciones son unchecked

6. ¿Qué permite el 'multi-catch' introducido en Java 7 (catch (IOException | SQLException e))?

- A) Capturar varios tipos de excepción no relacionados jerárquicamente en un único bloque catch, evitando duplicar código de manejo
- B) Capturar todas las excepciones posibles automáticamente sin especificarlas
- C) Ejecutar el bloque catch varias veces en secuencia
- D) Es solo válido para excepciones unchecked

7. ¿Qué es una excepción personalizada (custom exception) y cómo se crea típicamente?

- A) Una clase que extiende Exception (checked) o RuntimeException (unchecked) para representar un error específico del dominio de la aplicación
- B) Una excepción que solo puede lanzarse desde el método main
- C) Una excepción que no puede tener mensaje ni causa asociada
- D) Un tipo de anotación que reemplaza al try-catch

BigDecimal y precisión numérica

1. ¿Por qué se recomienda usar BigDecimal en lugar de double para cálculos monetarios?

- A) Porque double usa representación de punto flotante binario que introduce errores de redondeo; BigDecimal representa números decimales exactos con precisión arbitraria
- B) Porque double no puede almacenar números negativos
- C) Porque BigDecimal es más rápido en todas las operaciones aritméticas
- D) Porque double no permite más de 2 decimales

2. ¿Cuál es la forma recomendada de crear un BigDecimal a partir de un valor decimal exacto como 19.99?

- A) `new BigDecimal("19.99")` usando un String, para evitar imprecisión del double
- B) `new BigDecimal(19.99)` usando el double directamente, siempre es exacto
- C) `BigDecimal.valueOf(19.99f)` usando float
- D) No es posible crear BigDecimal con decimales

3. ¿Por qué comparar dos BigDecimal con `equals()` puede dar resultados inesperados, por ejemplo entre `new BigDecimal("2.0")` y `new BigDecimal("2.00")`?

- A) Porque `equals()` considera también la escala (número de decimales); para comparar solo el valor numérico se debe usar `compareTo()`
- B) Porque BigDecimal no sobrescribe `equals()`
- C) Porque BigDecimal siempre lanza excepción al comparar
- D) Porque `equals()` en BigDecimal compara referencias, no valores

4. ¿Qué método de BigDecimal se usa para redondear un valor a una cantidad específica de decimales con un modo de redondeo determinado?

- A) `setScale(decimales, RoundingMode.HALF_UP)` (u otro `RoundingMode`)
- B) `round(decimales)`
- C) `truncate(decimales)`
- D) `toFixed(decimales)`

5. ¿Por qué BigDecimal es inmutable, igual que String?

- A) Porque cada operación aritmética (`add`, `subtract`, `multiply`...) devuelve un nuevo objeto BigDecimal en lugar de modificar el original, garantizando consistencia y seguridad en entornos concurrentes
- B) Porque solo puede contener un dígito a la vez
- C) Porque no permite operaciones aritméticas en absoluto
- D) Porque BigDecimal está deprecado desde Java 17

Internacionalización (Locale, ResourceBundle)

1. ¿Qué clase representa una región geográfica, política o cultural específica para adaptar formatos y textos (i18n)?

- A) `java.util.Locale`
- B) `java.util.Region`
- C) `java.util.Country`
- D) `java.text.Zone`

2. ¿Cómo se cargan mensajes traducidos según el idioma usando ResourceBundle?

- A) `ResourceBundle.getBundle("nombreBase", locale)` busca automáticamente el archivo `.properties` más específico que coincida con el `Locale` dado, con fallback al bundle por defecto
- B) Se debe escribir manualmente un `if-else` para cada idioma soportado
- C) `ResourceBundle` solo soporta inglés y español
- D) Los archivos `.properties` deben estar compilados como clases `.class`

3. ¿Qué clase se usa para formatear números o monedas de acuerdo a un Locale específico?

- A) `java.text.NumberFormat`
- B) `java.lang.NumberParser`
- C) `java.util.Formatter.Locale`
- D) `java.text.LocaleFormat`

4. ¿Cómo se crea un Locale específico para español de México?

- A) `Locale.of("es", "MX")` (o el constructor equivalente `new Locale("es","MX")` en versiones previas)
- B) `new Locale("MX-es")`
- C) `Locale.create("spanish", "mexico")`
- D) `Locale.MEXICO_SPANISH`

5. ¿Qué convención de nombres de archivo sigue un ResourceBundle basado en propiedades para el Locale francés (fr_FR), con nombre base 'mensajes'?

- A) `mensajes_fr_FR.properties`
- B) `mensajes-fr-FR.properties`
- C) `fr_FR_mensajes.properties`
- D) `mensajes.fr.FR.properties`

Patrones de diseño (Strategy)

1. ¿Qué problema resuelve el patrón de diseño Strategy?

- A) Permite definir una familia de algoritmos intercambiables, encapsulando cada uno en una implementación separada, de modo que el algoritmo pueda variar independientemente del código que lo usa
- B) Permite crear una única instancia global de una clase
- C) Convierte una interfaz incompatible en otra que el cliente espera
- D) Permite añadir responsabilidades a un objeto dinámicamente envolviéndolo

2. En Java moderno, ¿qué característica del lenguaje permite implementar el patrón Strategy de forma muy concisa sin crear clases separadas para cada algoritmo?

- A) Expresiones lambda que implementan una interfaz funcional que representa la estrategia
- B) Los records
- C) Los enums simples
- D) Los bloques de texto

3. ¿Cuál es una ventaja de usar Strategy frente a múltiples bloques if-else o switch para seleccionar un algoritmo?

- A) Facilita añadir nuevas estrategias sin modificar el código existente que las usa (principio abierto/cerrado)
- B) Hace que el código se ejecute más rápido siempre
- C) Elimina la necesidad de interfaces
- D) Convierte el código automáticamente en concurrente

4. En el patrón Strategy, ¿qué rol cumple la clase 'Contexto'?

- A) Mantiene una referencia a una estrategia (interfaz común) y delega en ella la ejecución del algoritmo, sin conocer los detalles de la implementación concreta
- B) Es la implementación concreta de un algoritmo específico
- C) Es la interfaz que define el contrato del algoritmo
- D) Es una clase estática que no puede cambiar en tiempo de ejecución

Clases anidadas e internas

1. ¿Qué es una clase interna (inner class) no estática en Java?

- A) Una clase definida dentro de otra clase que mantiene una referencia implícita a una instancia de la clase contenedora, permitiendo acceder a sus miembros de instancia
- B) Una clase que solo puede definirse dentro de un método main
- C) Una clase que no puede tener métodos propios
- D) Un sinónimo exacto de clase estática anidada

2. ¿Cuál es la diferencia principal entre una clase anidada 'static' y una clase interna (no estática)?

- A) La clase anidada static no necesita una instancia de la clase externa para crearse y no tiene acceso implícito a los miembros de instancia de esta; la clase interna sí requiere esa instancia
- B) La clase anidada static no puede tener métodos
- C) Ambas son exactamente iguales, solo cambia el nombre
- D) Solo la clase interna puede ser private

3. ¿Qué es una clase local (local class) en Java?

- A) Una clase definida dentro del cuerpo de un método, visible solo dentro de ese método
- B) Una clase que solo puede usarse en el paquete java.util
- C) Una clase que no puede implementar interfaces
- D) Un sinónimo de clase anónima

4. ¿Puede una clase interna no estática acceder directamente a variables private de la clase externa que la contiene?

- A) Sí, las clases internas tienen acceso completo a todos los miembros (incluidos private) de la clase externa que las contiene
- B) No, nunca puede acceder a miembros private de la clase externa
- C) Solo si ambas clases están en el mismo archivo pero en paquetes distintos
- D) Solo si la clase interna es también private

API de fecha y hora (java.time)

- 1. ¿Qué clase del paquete java.time representa una fecha sin hora, como '2026-07-07'?**
 - A) LocalDate
 - B) LocalDateTime
 - C) LocalTime
 - D) Instant
- 2. ¿Qué clase representa una fecha y hora sin información de zona horaria?**
 - A) LocalDateTime
 - B) ZonedDateTime
 - C) Instant
 - D) OffsetDateTime
- 3. ¿Son mutables los objetos de las clases del paquete java.time como LocalDate y LocalDateTime?**
 - A) No, son inmutables; métodos como plusDays() devuelven una nueva instancia en lugar de modificar el objeto original
 - B) Sí, son completamente mutables como Calendar
 - C) Solo LocalDate es inmutable, LocalDateTime no
 - D) Depende de si se usa un builder
- 4. ¿Qué clase se usa para representar una cantidad de tiempo basada en fechas (años, meses, días), como la diferencia entre dos LocalDate?**
 - A) Period
 - B) Duration
 - C) Instant
 - D) Interval
- 5. ¿Qué método se usa para calcular la diferencia entre dos LocalDate en días?**
 - A) ChronoUnit.DAYS.between(fecha1, fecha2)
 - B) fecha1.diff(fecha2)
 - C) fecha1.minus(fecha2)
 - D) Period.diasEntre(fecha1, fecha2)

Autoboxing, unboxing y wrapper classes

1. ¿Qué es el 'unboxing' en Java?

- A) La conversión automática de un objeto wrapper (como Integer) a su tipo primitivo correspondiente (como int)
- B) El proceso de crear un array de wrappers
- C) Una técnica para deshabilitar el autoboxing
- D) Un método para serializar objetos

2. ¿Qué ocurre si se intenta hacer unboxing de un valor Integer que es null, por ejemplo al asignarlo a un int?

- A) Se lanza NullPointerException en tiempo de ejecución
- B) Se asigna automáticamente el valor 0
- C) El compilador lo rechaza en compilación
- D) Se lanza ClassCastException

3. ¿Qué imprime este código?

```
Integer a = 100;  
Integer b = 100;  
System.out.println(a == b);  
Integer c = 200;  
Integer d = 200;  
System.out.println(c == d);
```

```
Integer a = 100;  
Integer b = 100;  
System.out.println(a == b);  
Integer c = 200;  
Integer d = 200;  
System.out.println(c == d);
```

- A) true y false
- B) true y true
- C) false y false
- D) false y true

4. ¿Cuál es el método recomendado para convertir un String a un int de forma directa, sin pasar por un objeto Integer?

- A) Integer.parseInt(texto)
- B) Integer.valueOf(texto).getPrimitive()
- C) new Integer(texto).toInt()
- D) int.parse(texto)

Módulos Java (JPMS)

1. ¿Qué archivo declara un módulo en el sistema de módulos de Java (JPMS, introducido en Java 9)?
 - A) module-info.java
 - B) module.xml
 - C) package-info.java
 - D) moduleconfig.properties
2. ¿Qué directiva dentro de module-info.java indica que un módulo depende de otro módulo?
 - A) requires nombreModulo;
 - B) imports nombreModulo;
 - C) depends nombreModulo;
 - D) uses nombreModulo;
3. ¿Qué directiva permite que otros módulos accedan a las clases públicas de un paquete específico?
 - A) exports nombrePaquete;
 - B) public nombrePaquete;
 - C) open nombrePaquete;
 - D) expose nombrePaquete;
4. ¿Cuál es uno de los principales beneficios del sistema de módulos (JPMS) respecto al classpath tradicional?
 - A) Permite encapsulamiento fuerte a nivel de módulo: un paquete no exportado no es accesible desde fuera, incluso si sus clases son públicas
 - B) Elimina por completo la necesidad de compilar el código
 - C) Hace que todos los paquetes sean automáticamente públicos entre módulos
 - D) Reemplaza completamente a Maven y Gradle
5. ¿Qué directiva se usa en module-info.java para declarar que un módulo abre un paquete a la reflexión (por ejemplo para frameworks como Hibernate)?
 - A) opens nombrePaquete;
 - B) reflect nombrePaquete;
 - C) unlock nombrePaquete;
 - D) grant nombrePaquete;

Respuestas y explicaciones

Fundamentos JVM/JDK

1. ¿Cuál es la diferencia principal entre JDK, JRE y JVM?

Respuesta correcta: A) JDK incluye herramientas de desarrollo (javac, etc.) y el JRE; el JRE contiene la JVM y las librerías necesarias para ejecutar; la JVM ejecuta el bytecode

El JDK (Java Development Kit) contiene compilador, herramientas y el JRE. El JRE (Java Runtime Environment) trae la JVM y librerías estándar para ejecutar. La JVM interpreta/compila JIT el bytecode .class.

2. ¿Qué extensión tiene el archivo generado tras compilar un .java con javac?

Respuesta correcta: A) .class

javac genera archivos .class que contienen bytecode, ejecutado luego por la JVM.

3. ¿Qué instrucción usarías para ejecutar directamente un archivo fuente Java de un solo archivo sin compilar manualmente (característica desde Java 11)?

Respuesta correcta: A) java MiArchivo.java

Desde JEP 330 (Java 11) se puede ejecutar 'java MiArchivo.java' para lanzar un programa de un solo archivo fuente sin compilación explícita.

4. ¿Qué hace el Garbage Collector (recolector de basura) en Java?

Respuesta correcta: A) Libera automáticamente la memoria de objetos que ya no tienen referencias alcanzables, sin que el programador deba liberarla manualmente

El Garbage Collector es un proceso de la JVM que identifica y libera memoria ocupada por objetos que ya no son alcanzables desde ninguna referencia activa del programa.

5. ¿Qué firma debe tener obligatoriamente el método main para que la JVM pueda ejecutar un programa?

Respuesta correcta: A) public static void main(String[] args)

La JVM busca exactamente 'public static void main(String[] args)' (o su variante con varargs String... args) como punto de entrada del programa.

6. ¿Qué es JShell, introducido en Java 9?

Respuesta correcta: A) Una herramienta REPL (Read-Eval-Print Loop) que permite ejecutar fragmentos de código Java de forma interactiva sin crear un proyecto completo

JShell es una consola interactiva (REPL) para probar expresiones, declaraciones y fragmentos de Java sin necesidad de compilar un proyecto completo.

Primitivos, casting y var

1. ¿Cuáles son los 8 tipos primitivos de Java?

Respuesta correcta: A) byte, short, int, long, float, double, char, boolean

Los 8 tipos primitivos son: byte, short, int, long, float, double, char y boolean. String no es primitivo.

2. ¿Qué imprime este código?

```
int x = 130;  
byte b = (byte) x;  
System.out.println(b);
```

Respuesta correcta: A) -126

byte es de 8 bits (rango -128 a 127). 130 se desborda: $130 - 256 = -126$ debido al overflow al hacer narrowing cast.

3. ¿Cuál de las siguientes declaraciones con 'var' NO compila?

Respuesta correcta: A) var x; (sin inicializador)

'var' requiere inicialización en la misma línea porque el compilador infiere el tipo a partir del valor asignado; no puede usarse sin inicializador ni como tipo de campo/parámetro.

4. ¿Se puede usar 'var' para declarar un atributo (campo) de una clase?

Respuesta correcta: A) No, 'var' solo es válido para variables locales (dentro de métodos, bloques, for, try-with-resources)

'var' (inferencia de tipo local, LVTI) solo aplica a variables locales, no a campos de clase, parámetros de método o tipos de retorno.

5. ¿Qué es el 'autoboxing' en Java?

Respuesta correcta: A) La conversión automática que realiza el compilador de un tipo primitivo a su clase wrapper correspondiente (por ejemplo, int a Integer)

El autoboxing convierte automáticamente tipos primitivos en sus wrappers (int→Integer, double→Double, etc.) cuando el contexto lo requiere, por ejemplo al añadir un int a una List<Integer>.

6. ¿Qué imprime este código?

```
double d = 9.99;  
int i = (int) d;  
System.out.println(i);
```

Respuesta correcta: A) 9

El casting explícito de double a int simplemente trunca la parte decimal (no redondea), por lo que 9.99 se convierte en 9.

7. ¿Cuál de las siguientes asignaciones requiere un cast explícito para compilar?

Respuesta correcta: A) int x = (int) 3.5; (double a int)

Las conversiones de un tipo 'más ancho' a uno 'más estrecho' (narrowing), como double a int, requieren cast explícito porque pueden perder precisión. Las conversiones de ensanchamiento (widening) como int→long son implícitas.

Operadores y control de flujo

1. ¿Qué imprime el siguiente switch expression?

```
int dia = 3;  
String r = switch(dia){  
case 1,7 -> "Fin de semana";  
default -> "Entre semana";  
};  
System.out.println(r);
```

Respuesta correcta: A) Entre semana

Con la nueva sintaxis de switch expression (flechas), múltiples valores se agrupan con comas. Como dia=3 no coincide con 1 ni 7, cae en default.

2. ¿Qué palabra clave se usa en un switch expression para devolver un valor calculado dentro de un bloque {}?

Respuesta correcta: A) yield

'yield' se usa dentro de un bloque {} de un switch expression para producir el valor resultante.

3. ¿Cuál es el resultado de: System.out.println(5 / 2); y luego System.out.println(5.0 / 2);?

Respuesta correcta: A) 2 y 2.5

La división entera (int/int) trunca el decimal, da 2. Si al menos un operando es double, el resultado es double, da 2.5.

4. ¿Qué diferencia hay entre los operadores '&&' y '&' cuando se usan con dos expresiones booleanas?

Respuesta correcta: A) '&&' usa cortocircuito (no evalúa el segundo operando si el primero ya determina el resultado); '&' siempre evalúa ambos operandos

El operador '&&' (AND lógico con cortocircuito) no evalúa el segundo operando si el primero ya es false; '&' (AND bit a bit/lógico) siempre evalúa ambos lados, lo cual es relevante si el segundo operando tiene efectos secundarios.

5. ¿Qué imprime este bucle?

```
for(int i = 0; i < 3; i++){  
    if(i == 1) continue;  
    System.out.print(i);  
}
```

Respuesta correcta: A) 02

continue salta directamente a la siguiente iteración sin ejecutar el resto del cuerpo del bucle; cuando i==1 se omite el print, por lo que se imprime 0 y luego 2.

6. ¿Es obligatorio el 'break' al final de cada rama en un switch statement clásico (con dos puntos ':')?

Respuesta correcta: A) No es obligatorio sintácticamente, pero sin él el flujo 'cae' (fall-through) a la siguiente rama, lo cual suele ser un error común

En un switch clásico, si no se coloca 'break', la ejecución continúa (fall-through) hacia el siguiente case, ejecutando su código también, lo cual suele ser una fuente de errores si no es intencional.

Arrays y varargs

1. ¿Cuál declaración de un método varargs es válida?

Respuesta correcta: A) void metodo(String... nombres)

La sintaxis correcta de varargs es 'Tipo... nombre'. Solo puede haber un parámetro varargs y debe ser el último de la lista.

2. ¿Qué ocurre si un método tiene un parámetro varargs y otro sobrecargado con array explícito del mismo tipo?

Respuesta correcta: A) Compila; Java prefiere la coincidencia exacta (el método con array explícito) antes que expandir varargs

La resolución de sobrecarga en Java prioriza la coincidencia sin varargs (aplicabilidad estricta) antes de considerar métodos varargs.

3. `int[] a = new int[3];` ¿cuál es el valor por defecto de cada elemento?

Respuesta correcta: A) 0

Los arrays de tipos primitivos numéricos se inicializan por defecto a 0 (boolean a false, char a '\u0000', referencias a null).

4. ¿Qué imprime este código?

```
int[][] matriz = {{1,2},{3,4,5}};  
System.out.println(matriz[1].length);
```

Respuesta correcta: A) 3

Java permite arrays multidimensionales 'irregulares' (jagged arrays), donde cada fila puede tener distinto tamaño. `matriz[1]` es `{3,4,5}`, cuyo `length` es 3.

5. ¿Qué método de la clase Arrays permite convertir un array en una representación de texto legible, como "[1, 2, 3]"?

Respuesta correcta: A) Arrays.toString(array)

`Arrays.toString()` genera una representación legible de un array unidimensional. Para arrays multidimensionales existe `Arrays.deepToString()`.

6. ¿Qué ocurre al intentar acceder a un índice fuera de rango de un array, por ejemplo array[10] en un array de tamaño 5?

Respuesta correcta: A) Se lanza ArrayIndexOutOfBoundsException en tiempo de ejecución

Acceder a un índice inválido de un array lanza ArrayIndexOutOfBoundsException en tiempo de ejecución; el compilador no puede detectar este error en general.

Strings y bloques de texto

1. ¿Por qué se dice que String es immutable en Java?

Respuesta correcta: A) Porque una vez creado un objeto String, su contenido no puede modificarse; operaciones como concat() devuelven un nuevo objeto

Los objetos String son inmutables: cualquier método que 'modifica' un String (concat, replace, toUpperCase...) realmente crea y devuelve un nuevo objeto.

2. ¿Qué imprime?

```
String a = "hola";
```

```
String b = "hola";
```

```
System.out.println(a == b);
```

Respuesta correcta: A) true

Los literales String se almacenan en el String pool. Ambas variables referencian el mismo objeto interno, así que '==' (comparación de referencia) da true.

3. ¿Cuál es la sintaxis correcta para iniciar un bloque de texto (text block, Java 15+)?

Respuesta correcta: A) Triple comillas dobles """" seguidas de salto de línea

Un text block comienza con tres comillas dobles y un salto de línea inmediatamente después, terminando también con tres comillas dobles.

4. ¿Qué método se usa para comparar el contenido de dos Strings (no la referencia)?

Respuesta correcta: A) equals()

'equals()' compara el contenido carácter por carácter; '==' compara si ambas referencias apuntan al mismo objeto en memoria.

5. ¿Por qué se recomienda StringBuilder en lugar de concatenar Strings con '+' dentro de un bucle con muchas iteraciones?

Respuesta correcta: A) Porque cada concatenación con '+' crea un nuevo objeto String (dado que String es immutable), lo que es ineficiente; StringBuilder modifica un buffer mutable interno

Al ser String immutable, cada '+' genera un nuevo objeto, desperdiciando memoria y tiempo en bucles largos. StringBuilder usa un array de caracteres mutable interno, evitando esas creaciones repetidas.

6. ¿Qué imprime?

```
String s = " Hola Mundo ";
```

```
System.out.println(s.strip().length());
```

Respuesta correcta: A) 10

strip() (Java 11+) elimina espacios en blanco al inicio y al final. "Hola Mundo" sin los espacios circundantes tiene 10 caracteres.

7. ¿Qué método de String permite dividir el texto en un array usando un delimitador, por ejemplo una coma?

Respuesta correcta: A) split(regex)

split(String regex) divide el String en un array de substrings usando la expresión regular indicada como delimitador.

POO: clases, constructores y encapsulamiento

1. ¿Qué ocurre si una clase no define ningún constructor explícito?

Respuesta correcta: A) El compilador genera automáticamente un constructor por defecto sin argumentos

Si no se declara ningún constructor, el compilador provee uno por defecto (sin parámetros) que llama implícitamente a `super()`.

2. ¿Qué significa que un atributo sea 'static'?

Respuesta correcta: A) Pertenece a la clase y es compartido por todas las instancias, en lugar de pertenecer a cada objeto individual

Un miembro `static` pertenece a la clase, no a instancias individuales; existe una sola copia compartida por todos los objetos.

3. ¿Cuál es el propósito principal del modificador 'final' aplicado a una variable?

Respuesta correcta: A) Impedir que la variable sea reasignada después de su primera asignación

'final' en una variable impide reasignación tras la inicialización. En una clase impide herencia; en un método impide `override`.

4. ¿Puede una clase declarada como 'final' ser extendida (heredada)?

Respuesta correcta: A) No, una clase final no puede tener subclases

'final' en una clase prohíbe que otras clases la extiendan, por ejemplo `java.lang.String` es final.

5. ¿Qué es un bloque de inicialización estático (static initializer block) y cuándo se ejecuta?

Respuesta correcta: A) Un bloque `static { ... }` que se ejecuta una única vez, cuando la clase se carga por primera vez en la JVM, antes de crear cualquier instancia

Un bloque `static {}` se ejecuta una sola vez al cargar la clase en memoria, útil para inicializar campos `static` complejos, antes de cualquier instanciación.

6. ¿Cuál es el propósito principal del encapsulamiento en POO?

Respuesta correcta: A) Ocultar el estado interno de un objeto y exponer solo una interfaz pública controlada (getters/setters), protegiendo la integridad de los datos

El encapsulamiento oculta los detalles internos (atributos generalmente `private`) y expone un comportamiento controlado a través de métodos públicos, protegiendo la consistencia del objeto.

7. ¿Qué hace la palabra clave 'this' dentro de un constructor cuando se usa como `this(...)`?

Respuesta correcta: A) Invoca a otro constructor sobrecargado de la misma clase (constructor chaining), y debe ser la primera línea del constructor

`this(...)` se usa para llamar a otro constructor de la misma clase (encadenamiento de constructores), y debe ser obligatoriamente la primera sentencia dentro del constructor.

Herencia y polimorfismo

1. ¿Qué es 'overriding' (sobrescritura) de métodos?

Respuesta correcta: A) Redefinir en una subclase un método con la misma firma que el de la superclase, cambiando su implementación

El overriding ocurre cuando una subclase redefine un método heredado con la misma firma; el overloading es tener varios métodos con el mismo nombre pero distinta lista de parámetros.

2. ¿Qué palabra clave permite invocar el constructor o método de la superclase desde una subclase?

Respuesta correcta: A) `super`

'super' referencia la superclase; `super()` invoca su constructor y `super.metodo()` invoca su implementación de un método.

3. Dado:

```
class Animal { void sonido(){ System.out.println("..."); } }  
class Perro extends Animal { void sonido(){ System.out.println("Guau"); } }  
Animal a = new Perro();  
a.sonido();
```

¿Qué se imprime?

Respuesta correcta: A) Guau

El polimorfismo en tiempo de ejecución (dynamic dispatch) hace que se ejecute la versión sobrescrita del objeto real (Perro), no la del tipo de referencia declarado (Animal).

4. Java permite herencia múltiple de clases pero sí de interfaces. ¿Verdadero o falso?

Respuesta correcta: A) Falso: Java NO permite herencia múltiple de clases (solo una superclase), pero Sí permite implementar múltiples interfaces

Una clase solo puede extender una única superclase (extends), pero puede implementar múltiples interfaces (implements A, B, C).

5. ¿Qué modificador de acceso permite que un miembro sea visible solo dentro del mismo paquete y en subclases de otros paquetes?

Respuesta correcta: A) protected

'protected' permite el acceso desde el mismo paquete y también desde subclases ubicadas en paquetes distintos. El modificador 'default' (sin palabra clave) solo permite acceso dentro del mismo paquete.

6. ¿Puede un método sobrescrito (override) en una subclase declarar una excepción checked más amplia que la del método original de la superclase?

Respuesta correcta: A) No, el método sobrescrito solo puede declarar la misma excepción checked, una subclase de ella, o ninguna; nunca una más amplia

Una regla de overriding es que el método no puede lanzar excepciones checked nuevas o más amplias que las declaradas en el método de la superclase, para preservar la sustituibilidad de Liskov.

7. ¿Qué es el 'binding estático' (static binding) frente al 'binding dinámico' (dynamic binding)?

Respuesta correcta: A) El binding estático resuelve en compilación qué método se llama (métodos static, private, final, y sobrecarga); el binding dinámico resuelve en tiempo de ejecución (métodos sobrescritos vía polimorfismo)

Los métodos static, private y final (y la resolución de sobrecarga) se enlazan en compilación (static binding); los métodos de instancia sobrescritos se resuelven en runtime según el tipo real del objeto (dynamic binding/dispatch).

Interfaces y clases abstractas

1. ¿Qué permiten los métodos 'default' en una interfaz (desde Java 8)?

Respuesta correcta: A) Proveer una implementación concreta dentro de la interfaz que las clases implementadoras pueden usar o sobrescribir

Los métodos default tienen cuerpo dentro de la interfaz, permitiendo evolucionar interfaces sin romper implementaciones existentes.

2. ¿Puede una clase abstracta tener constructores?

Respuesta correcta: A) Sí, aunque no se pueda instanciar directamente, sus constructores se ejecutan al instanciar subclases concretas

Las clases abstractas pueden tener constructores, campos y métodos concretos; se invocan mediante super() al construir una subclase concreta.

3. ¿Qué es una clase anónima en Java?

Respuesta correcta: A) Una clase sin nombre que se define e instancia en un único paso, útil para implementar una interfaz o extender una clase de forma puntual

Una clase anónima combina la declaración de una subclase (o implementación de interfaz) y su instanciación en una sola expresión, sin nombre propio.

4. ¿Cuántos métodos abstractos puede tener como máximo una interfaz funcional?

Respuesta correcta: A) Exactamente uno (además de métodos default/static)

Una interfaz funcional (anotada opcionalmente con `@FunctionalInterface`) debe declarar exactamente un método abstracto, lo que permite implementarla con una expresión lambda.

5. ¿Puede una interfaz declarar campos (variables)?

Respuesta correcta: A) Sí, pero son implícitamente public static final (constantes), no se pueden tener campos de instancia mutables

Cualquier campo declarado en una interfaz es implícitamente `public static final`, es decir, una constante compartida, no un campo de instancia.

6. ¿Qué son los métodos 'static' dentro de una interfaz (desde Java 8)?

Respuesta correcta: A) Métodos con implementación que pertenecen a la interfaz misma y se invocan como `NombreInterfaz.metodo()`, no se heredan a las clases implementadoras

Los métodos `static` en interfaces se invocan sobre el nombre de la interfaz (como utilidades relacionadas), y no se heredan ni pueden sobrescribirse en las clases implementadoras.

7. Si una clase implementa dos interfaces que definen el mismo método default con distinta implementación, ¿qué ocurre?

Respuesta correcta: A) La clase debe sobrescribir explícitamente ese método para resolver la ambigüedad, o el código no compila

Ante un conflicto de métodos default heredados de múltiples interfaces, el compilador exige que la clase implementadora sobrescriba el método explícitamente para eliminar la ambigüedad.

Enums

1. ¿Pueden los enums en Java tener constructores, campos y métodos?

Respuesta correcta: A) Sí, un enum es una clase especial que puede tener constructores (siempre implícitamente privados), campos y métodos, incluso implementar interfaces

Los enums son clases especiales; pueden tener campos, constructores (implícitamente `private`) y métodos, además de implementar interfaces y sobrescribir métodos por constante.

2. ¿Qué método permite obtener la posición ordinal (basada en 0) de una constante enum?

Respuesta correcta: A) `ordinal()`

`ordinal()` devuelve la posición de la constante según el orden de declaración, comenzando en 0. `name()` devuelve su nombre como `String`.

3. ¿Se puede usar un enum en un switch statement/expression sin prefijo del nombre de la clase en los 'case'?

Respuesta correcta: A) Sí, solo se usa el nombre de la constante (ej. `case LUNES ->`)

En un switch sobre un enum, los case usan directamente el nombre de la constante sin calificar con el nombre de la clase.

4. ¿Qué método permite obtener todas las constantes de un enum como un array?

Respuesta correcta: A) `NombreEnum.values()`

El método `static` implícito `values()` devuelve un array con todas las constantes del enum en el orden en que fueron declaradas.

5. ¿Puede un enum implementar un cuerpo de clase distinto para cada constante (constant-specific class body)?

Respuesta correcta: A) Sí, cada constante puede sobrescribir un método abstracto declarado en el enum con su propia implementación

Un enum puede declarar un método abstracto y cada constante puede proveer su propio cuerpo de implementación entre llaves, permitiendo comportamiento distinto por constante.

6. ¿Qué excepción se lanza si se llama a `valueOf()` de un enum con un nombre que no corresponde a ninguna constante?

Respuesta correcta: A) `IllegalArgumentException`

`NombreEnum.valueOf("texto")` lanza `IllegalArgumentException` si el texto no coincide exactamente con el nombre de ninguna constante declarada.

Records

1. ¿Qué genera automáticamente el compilador al declarar un record?

Respuesta correcta: A) Constructor canónico, métodos de acceso (accessors) con el nombre del componente, `equals()`, `hashCode()` y `toString()`

Un record genera automáticamente: constructor canónico, accessors (con el mismo nombre del campo, sin prefijo 'get'), `equals()`, `hashCode()` y `toString()`. No genera setters porque los records son inmutables.

2. ¿Son mutables los componentes de un record?

Respuesta correcta: A) No, todos los campos de un record son implícitamente final e inmutables

Los records son inmutables por diseño: sus campos son implícitamente `private final`, sin setters generados.

3. ¿Qué es un 'constructor compacto' en un record?

Respuesta correcta: A) Una forma de escribir el constructor canónico sin repetir la lista de parámetros, útil para validar o normalizar valores antes de la asignación implícita

El constructor compacto se escribe como `'public NombreRecord { ... validaciones ... }'` sin paréntesis de parámetros; el compilador asigna los campos automáticamente al final.

4. ¿Puede un record implementar interfaces?

Respuesta correcta: A) Sí, un record puede implementar una o varias interfaces, igual que una clase normal

Un record puede implementar cualquier interfaz (implements), aunque no puede extender otra clase, ya que implícitamente ya extiende `java.lang.Record`.

5. ¿Puede un record declarar campos de instancia adicionales fuera de los componentes definidos en su cabecera?

Respuesta correcta: A) No, un record solo puede tener los campos correspondientes a sus componentes declarados; sí puede tener campos static adicionales

Los records solo permiten campos de instancia correspondientes a los componentes de su cabecera (para mantener la transparencia de datos), aunque sí pueden declarar campos static adicionales.

6. ¿Qué genera el compilador si defines explícitamente el accessor de un componente de un record, por ejemplo `public int x() { return x * -1; }`?

Respuesta correcta: A) Usa la implementación personalizada en lugar del accessor generado automáticamente, permitiendo modificar qué se devuelve

Se puede sobrescribir explícitamente el método accessor de un componente; el compilador usará esa implementación personalizada en lugar de generar la trivial por defecto.

Sealed classes y pattern matching

1. ¿Para qué sirve la palabra clave 'sealed' en una clase o interfaz (Java 17)?

Respuesta correcta: A) Para restringir explícitamente qué clases pueden extenderla o implementarla, usando la cláusula 'permits'

'sealed' junto con 'permits' limita el conjunto de subtipos autorizados, permitiendo un control exhaustivo (útil junto a pattern matching en switch).

2. Una subclase de una clase sealed, ¿qué modificador debe declarar obligatoriamente?

Respuesta correcta: A) final, sealed o non-sealed

Cada subclase permitida de un tipo sealed debe declararse explícitamente como final, sealed (para seguir restringiendo) o non-sealed (para abrir la jerarquía de nuevo).

3. ¿Qué permite el pattern matching for instanceof (Java 16+)?

Respuesta correcta: A) Combinar la comprobación instanceof con la asignación a una variable en una sola expresión, evitando el cast manual: if (obj instanceof String s) {...}

Pattern matching for instanceof permite escribir 'if (obj instanceof String s)' y usar 's' ya tipado y con cast implícito dentro del bloque, sin necesidad de castear manualmente.

4. En Java 21, ¿qué aporta el pattern matching para switch sobre records (record patterns)?

Respuesta correcta: A) Permite desestructurar los componentes de un record directamente en el 'case' (ej. case Punto(int x, int y) -> ...)

Los record patterns (Java 21) permiten deconstruir un record en el propio patrón del case, extrayendo sus componentes como variables locales.

5. ¿Pueden las clases permitidas ('permits') de una clase sealed estar en un paquete diferente?

Respuesta correcta: A) Sí, pero solo si pertenecen al mismo módulo, o si están en el mismo paquete cuando no se usan módulos

Las subclases permitidas de un tipo sealed deben pertenecer al mismo módulo (en un proyecto modularizado) o, si no hay módulos, al mismo paquete que la clase sealed.

6. ¿Qué ventaja aporta un switch exhaustivo sobre una jerarquía sealed junto con pattern matching?

Respuesta correcta: A) El compilador puede verificar que se cubren todos los subtipos posibles, sin necesitar una rama 'default', mejorando la seguridad en compilación

Como el conjunto de subtipos de una clase/interfaz sealed es cerrado y conocido, el compilador puede verificar exhaustividad en un switch pattern matching sin exigir una rama default.

7. ¿Qué patrón usarías en un case para descomponer un record Punto(int x, int y) y aplicar una guarda adicional 'cuando x sea positivo' (Java 21, guarded patterns)?

Respuesta correcta: A) case Punto(int x, int y) when x > 0 -> ...

Java 21 introduce las guarded patterns con la palabra clave 'when' para añadir una condición booleana adicional al patrón deconstruido dentro de un case.

Generics y covarianza

1. ¿Qué problema resuelven los Generics en Java?

Respuesta correcta: A) Proveen seguridad de tipos en tiempo de compilación para colecciones y clases, evitando ClassCastException en tiempo de ejecución y castings manuales

Los generics permiten parametrizar tipos (List<String>, por ejemplo), detectando errores de tipo en compilación en vez de en runtime.

2. ¿Qué significa el comodín '? extends Number' en una declaración de generics?

Respuesta correcta: A) Acepta el tipo Number o cualquiera de sus subtipos (útil para lectura/producción de datos, 'Producer Extends')

'? extends Number' es un wildcard covariante: acepta Number o subtipos. Sigue el principio PECS (Producer Extends, Consumer Super); con extends solo puedes leer, no añadir (excepto null).

3. ¿Por qué List<String> NO es un subtipo de List<Object> en Java (a diferencia de los arrays)?

Respuesta correcta: A) Porque los generics son invariantes: aunque String sea subtipo de Object, List<String> y List<Object> son tipos distintos sin relación de subtipo entre sí

A diferencia de los arrays (covariantes, lo que puede causar ArrayStoreException en runtime), los generics son invariantes por diseño para mantener la seguridad de tipos en compilación.

4. ¿Qué es 'type erasure' (borrado de tipos) en Java?

Respuesta correcta: A) El proceso por el cual la información de tipo genérico se elimina en tiempo de compilación, quedando solo el tipo raw (u Object) en el bytecode

Debido a type erasure, la JVM no conserva los parámetros de tipo en tiempo de ejecución; List<String> y List<Integer> comparten la misma clase List en bytecode.

5. ¿Qué significa el comodín '? super Integer' en una declaración de generics?

Respuesta correcta: A) Acepta Integer o cualquiera de sus supertipos (útil para escribir/consumir datos, 'Consumer Super')

'? super Integer' es un wildcard contravariante: acepta Integer o cualquier supertipo suyo (como Number u Object), permitiendo añadir elementos Integer de forma segura (principio Consumer Super de PECS).

6. ¿Se puede crear un array de un tipo genérico parametrizado, como new List<String>[10]?

Respuesta correcta: A) No, Java no permite crear directamente arrays de tipos genéricos parametrizados debido al conflicto entre la covarianza de arrays y el borrado de tipos

Crear arrays genéricos parametrizados (new List<String>[10]) no es válido porque, combinado con la covarianza de arrays y el type erasure, podría romper la seguridad de tipos en tiempo de ejecución.

7. ¿Qué es un método genérico y cómo se declara su parámetro de tipo?

Respuesta correcta: A) Un método que declara su propio parámetro de tipo antes del tipo de retorno, por ejemplo: public <T> T primero(List<T> lista)

Un método genérico declara su propio parámetro de tipo entre '<>' justo antes del tipo de retorno, independientemente de si la clase contenedora es o no genérica.

Comparable y Comparator

1. ¿Cuál es la diferencia principal entre Comparable y Comparator?

Respuesta correcta: A) Comparable se implementa en la propia clase para definir su orden natural (compareTo); Comparator se define externamente y permite múltiples estrategias de ordenación (compare)

Comparable define el orden natural de un objeto mediante compareTo(); Comparator es una interfaz separada para definir órdenes alternativos mediante compare(a,b), útil para múltiples criterios.

2. ¿Qué valor devuelve compareTo() si el objeto actual es 'mayor' que el que se compara?

Respuesta correcta: A) Un valor positivo

Por convención: negativo si el objeto es menor, 0 si es igual, positivo si es mayor. No se garantiza un valor exacto, solo el signo.

3. ¿Cómo se encadenan múltiples criterios de comparación usando Comparator (por ejemplo, por apellido y luego por nombre)?

Respuesta correcta: A) Comparator.comparing(Persona::getApellido).thenComparing(Persona::getNombre)

El método thenComparing() permite encadenar criterios de desempate adicionales tras el comparador principal.

4. ¿Qué ocurre si dos objetos son 'iguales' según equals() pero compareTo() no devuelve 0 para ellos?

Respuesta correcta: A) Se dice que la clase tiene un orden 'inconsistente con equals', lo cual puede causar comportamientos inesperados en colecciones ordenadas como TreeSet o TreeMap

Se recomienda que compareTo() sea consistente con equals(); si no lo es, colecciones basadas en orden como TreeSet/TreeMap pueden comportarse de forma sorprendente (por ejemplo, tratar como iguales elementos que equals() distingue).

5. ¿Cómo se invierte el orden producido por un Comparator existente, por ejemplo para ordenar de mayor a menor?

Respuesta correcta: A) comparator.reversed()

El método por defecto reversed() de Comparator devuelve un nuevo Comparator con el orden invertido respecto al original.

6. ¿Qué método estático de Comparator crea un comparador a partir de una función que extrae una clave comparable de cada elemento?

Respuesta correcta: A) Comparator.comparing(funcionExtractora)

Comparator.comparing(Function keyExtractor) crea un Comparator que ordena según el valor Comparable devuelto por esa función aplicada a cada elemento.

Lambdas e interfaces funcionales

1. ¿Cuál es la sintaxis correcta de una expresión lambda que recibe dos enteros y devuelve su suma?

Respuesta correcta: A) (a, b) -> a + b

La sintaxis de lambdas en Java es (parámetros) -> expresión_o_bloque. No se usa '=>' como en JavaScript.

2. ¿Qué interfaz funcional del paquete java.util.function representa una operación que recibe un argumento y devuelve un boolean?

Respuesta correcta: A) Predicate<T>

Predicate<T> tiene el método test(T t) que devuelve boolean, típicamente usado para filtrar (Stream.filter).

3. ¿Qué interfaz funcional representa una operación que NO recibe argumentos pero produce un resultado?

Respuesta correcta: A) Supplier<T>

Supplier<T> tiene el método get() sin parámetros que produce/devuelve un valor de tipo T, útil para provisión perezosa de valores.

4. ¿Puede una expresión lambda acceder y modificar una variable local externa capturada?

Respuesta correcta: A) No, solo puede leer variables locales externas que sean 'effectively final' (efectivamente finales); no puede reasignarlas

Las lambdas solo pueden capturar variables locales 'effectively final' (que no cambian tras su inicialización); intentar reasignarlas produce error de compilación.

5. ¿Qué interfaz funcional recibe un argumento y no devuelve ningún resultado (void)?

Respuesta correcta: A) Consumer<T>

Consumer<T> define el método accept(T t) que recibe un argumento y no retorna nada, útil para operaciones como forEach.

6. ¿Qué interfaz funcional recibe un argumento de tipo T y produce un resultado de tipo R distinto?

Respuesta correcta: A) Function<T,R>

Function<T,R> define apply(T t) devolviendo R, permitiendo transformar un tipo en otro. UnaryOperator<T> es un caso especial donde entrada y salida son del mismo tipo T.

7. ¿Puede una expresión lambda lanzar una excepción checked si la interfaz funcional que implementa no la declara en su método abstracto?

Respuesta correcta: A) No, si el método abstracto de la interfaz funcional no declara esa excepción checked, la lambda no puede lanzarla (no compila)

El cuerpo de una lambda debe respetar la firma (incluidas las excepciones checked declaradas) del método abstracto de la interfaz funcional que implementa; si el método no declara throws, la lambda no puede lanzar una checked exception.

Method references

1. ¿Cuál de estas es la sintaxis de una referencia a método estático?

Respuesta correcta: A) Clase::metodoEstatico

Referencia a método estático: Clase::metodo. Ejemplo: Integer.parseInt(s) equivale a s -> Integer.parseInt(s).

2. ¿Qué representa la referencia a método String::toUpperCase usada como Function<String,String>?

Respuesta correcta: A) Una referencia a un método de instancia sobre un tipo arbitrario, equivalente a s -> s.toUpperCase()

Cuando se usa Clase::metodoDeInstancia sin especificar objeto, el primer parámetro de la interfaz funcional se convierte en el receptor de la llamada: equivale a s -> s.toUpperCase().

3. ¿Cómo se escribe una referencia a constructor para crear instancias de ArrayList mediante un Supplier?

Respuesta correcta: A) ArrayList::new

Clase::new es la sintaxis de referencia a constructor, usable como Supplier<ArrayList> u otra interfaz funcional compatible con la firma del constructor.

4. ¿Cuál es la sintaxis de una referencia a un método de instancia sobre un objeto particular ya existente, por ejemplo 'obj::metodo'?

Respuesta correcta: A) objetoExistente::metodoDeInstancia — el objeto ya está fijado como receptor de la llamada

Cuando se usa una referencia sobre un objeto ya creado (objeto::metodo), ese objeto queda fijado como receptor, y los parámetros de la interfaz funcional corresponden únicamente a los argumentos del método.

5. ¿A qué expresión lambda equivale la referencia de método Math::max usada como BinaryOperator<Integer>?

Respuesta correcta: A) (a, b) -> Math.max(a, b)

Math::max sobre una interfaz que recibe dos argumentos y devuelve un resultado equivale a (a, b) -> Math.max(a, b), ya que max es un método estático de dos parámetros.

Streams API

1. ¿Qué caracteriza a los streams en Java respecto a las colecciones que los originan?

Respuesta correcta: A) Un stream no almacena datos propios; procesa datos de una fuente de forma perezosa (lazy) y puede consumirse una sola vez

Los streams no almacenan elementos; describen operaciones sobre una fuente de datos, se evalúan de forma perezosa y son de un solo uso.

2. ¿Cuál de las siguientes es una operación TERMINAL de Stream (no intermedia)?

Respuesta correcta: A) collect()

collect(), forEach(), count(), reduce() son operaciones terminales que disparan el procesamiento. filter, map, sorted, distinct son intermedias y perezosas.

3. ¿Qué hace el siguiente código?

```
List<String> nombres = List.of("Ana","Luis","Eva");  
nombres.stream().filter(n -> n.length()>3).forEach(System.out::println);
```

Respuesta correcta: A) Imprime solo "Luis" porque es el único nombre con más de 3 caracteres

filter conserva solo elementos cuyo length() sea mayor a 3. 'Ana'(3) y 'Eva'(3) no cumplen; solo 'Luis'(4) pasa el filtro.

4. ¿Qué clase se usa típicamente para acumular resultados de un Stream en una colección mediante collect(), como Collectors.toList()?

Respuesta correcta: A) java.util.stream.Collectors

La clase utilitaria Collectors provee fábricas de Collector como toList(), toSet(), toMap(), joining(), groupingBy(), etc.

5. ¿Qué hace `Collectors.groupingBy()`?

Respuesta correcta: A) Agrupa los elementos de un stream en un Map según una función clasificadora, devolviendo un `Map<clave, List<elementos>>`

`groupingBy(funcionClasificadora)` agrupa los elementos según el resultado de esa función, generando un Map donde cada clave mapea a una lista (u otro collector downstream) de elementos.

6. ¿Cuál es la diferencia entre `map()` y `flatMap()` en Streams?

Respuesta correcta: A) `map()` transforma cada elemento en otro elemento uno a uno; `flatMap()` transforma cada elemento en un stream y luego 'aplana' todos esos streams en uno solo

`flatMap` se usa cuando cada elemento produce un Stream (por ejemplo, una lista de listas), y combina/aplana todos esos streams internos en un único Stream resultante.

7. ¿Qué devuelve la operación terminal `reduce()` en un Stream?

Respuesta correcta: A) Combina los elementos del stream en un único resultado acumulado, aplicando repetidamente una función de combinación binaria

`reduce()` combina todos los elementos usando una función asociativa (por ejemplo, sumar), produciendo un único valor acumulado, normalmente envuelto en `Optional` si no se especifica un valor identidad.

8. ¿Cuál es la diferencia entre un stream secuencial y uno paralelo (`parallelStream()`)?

Respuesta correcta: A) El stream paralelo divide el procesamiento entre múltiples hilos usando el `ForkJoinPool` común, mientras que el secuencial procesa los elementos uno a uno en un solo hilo

`parallelStream()` (o `stream().parallel()`) reparte el trabajo entre varios hilos del `ForkJoinPool` común, lo que puede acelerar operaciones costosas en datasets grandes, pero introduce consideraciones de orden y efectos secundarios que no aplican al stream secuencial.

9. ¿Qué método de Stream permite limitar la cantidad de elementos procesados, por ejemplo a los primeros 5?

Respuesta correcta: A) `limit(5)`

`limit(n)` es una operación intermedia que trunca el stream para que contenga como máximo n elementos.

10. ¿Qué método permite obtener un `IntStream` de números consecutivos, por ejemplo del 1 al 10?

Respuesta correcta: A) `IntStream.rangeClosed(1, 10)`

`IntStream.rangeClosed(1, 10)` genera un stream de enteros del 1 al 10 inclusive; `IntStream.range(1, 10)` generaría del 1 al 9 (excluyendo el límite superior).

Optional

1. ¿Cuál es el propósito principal de la clase `Optional<T>`?

Respuesta correcta: A) Representar explícitamente la posible ausencia de un valor, evitando `NullPointerException` y forzando un manejo explícito del caso 'vacío'

`Optional<T>` es un contenedor que puede o no tener un valor, comunicando explícitamente en la firma del método que el resultado puede estar ausente.

2. ¿Qué hace `Optional.ofNullable(valor)` si 'valor' es null?

Respuesta correcta: A) Devuelve un Optional vacío (`Optional.empty()`) sin lanzar excepción

A diferencia de `Optional.of()` (que lanza NPE si el argumento es null), `ofNullable()` acepta null y produce un `Optional` vacío en ese caso.

3. ¿Cuál es una forma recomendada de obtener un valor por defecto de un `Optional` si está vacío?

Respuesta correcta: A) `optional.orElse(valorPorDefecto)`

`orElse(valorDefault)` devuelve el valor contenido si existe, o el valor por defecto indicado si el `Optional` está vacío. También existe `orElseGet()` y `orElseThrow()`.

4. ¿Qué hace `Optional.map(funcion)` si el `Optional` está vacío?

Respuesta correcta: A) Devuelve directamente un `Optional` vacío, sin aplicar la función

Si el `Optional` está vacío, `map()` simplemente devuelve otro `Optional` vacío sin invocar la función de transformación, evitando `NullPointerException`.

5. ¿Qué excepción lanza `optional.get()` si el `Optional` está vacío?

Respuesta correcta: A) `NoSuchElementException`

Llamar a `get()` sobre un `Optional` vacío lanza `NoSuchElementException`; por eso se recomienda usar `isPresent()/ifPresent()`, `orElse()`, o `orElseThrow()` en su lugar.

6. ¿Cuál es una buena práctica respecto al uso de `Optional` como tipo de un campo de clase o parámetro de método?

Respuesta correcta: A) Se recomienda evitarlo: `Optional` está pensado principalmente como tipo de retorno de métodos, no para campos ni parámetros

La convención de la comunidad Java (y de los propios diseñadores del API) es usar `Optional` principalmente como tipo de retorno para indicar ausencia posible de valor, evitando su uso en campos de clase, parámetros o colecciones.

Collections Framework

1. ¿Cuál es la diferencia principal entre `List`, `Set` y `Map` en el Collections Framework?

Respuesta correcta: A) `List` permite duplicados y mantiene orden de inserción; `Set` no permite duplicados; `Map` almacena pares clave-valor

`List`: colección ordenada indexada con duplicados permitidos. `Set`: no permite elementos duplicados. `Map`: asocia claves únicas a valores (no extiende `Collection`).

2. ¿Qué implementación de `List` garantiza acceso rápido $O(1)$ por índice, pero inserciones/eliminaciones costosas en el medio?

Respuesta correcta: A) `ArrayList`

`ArrayList` se respalda en un array, por lo que el acceso por índice es $O(1)$, pero insertar/eliminar en medio requiere desplazar elementos. `LinkedList` es más eficiente en inserciones/eliminaciones.

3. ¿Qué característica distingue a un `TreeMap` de un `HashMap`?

Respuesta correcta: A) `TreeMap` mantiene sus claves ordenadas (orden natural o `Comparator`); `HashMap` no garantiza ningún orden

`TreeMap` implementa `NavigableMap` y mantiene las claves ordenadas; `HashMap` ofrece acceso $O(1)$ promedio pero sin orden garantizado.

4. ¿Qué devuelve `List.of(1,2,3)` en cuanto a mutabilidad?

Respuesta correcta: A) Una lista inmutable; intentar añadir o eliminar elementos lanza `UnsupportedOperationException`

Los métodos de fábrica `List.of()`, `Set.of()`, `Map.of()` (Java 9+) devuelven colecciones inmutables; cualquier intento de modificarlas lanza `UnsupportedOperationException`.

5. ¿Qué debe sobrescribir correctamente una clase para usarse como clave en un `HashMap` de forma consistente?

Respuesta correcta: A) `equals()` y `hashCode()` (contrato de `Object`)

`HashMap` usa `hashCode()` para ubicar el bucket y `equals()` para verificar igualdad de claves; si no se sobrescriben correctamente y de forma consistente, el `Map` puede comportarse mal.

6. ¿Qué estructura de datos implementa típicamente una `Queue` de tipo FIFO (primero en entrar, primero en salir)?

Respuesta correcta: A) `LinkedList` (implementando `Queue`)

`LinkedList` implementa la interfaz `Queue`, permitiendo comportamiento FIFO mediante métodos como `offer()`, `poll()` y `peek()`.

7. ¿Qué diferencia hay entre Iterator y ListIterator?

Respuesta correcta: A) ListIterator extiende Iterator añadiendo la capacidad de recorrer en ambas direcciones y modificar elementos durante la iteración (add, set)

ListIterator añade métodos como hasPrevious(), previous(), set() y add(), permitiendo recorrer una List en ambos sentidos y modificarla durante la iteración, algo que Iterator (más genérico) no ofrece.

8. ¿Qué excepción se lanza al modificar una colección (por ejemplo, con remove()) directamente mientras se itera sobre ella con un for-each, sin usar el propio Iterator?

Respuesta correcta: A) ConcurrentModificationException

Modificar estructuralmente una colección mientras se itera con for-each (que usa un Iterator internamente) sin pasar por los métodos del propio Iterator provoca ConcurrentModificationException en la mayoría de implementaciones.

Concurrencia y virtual threads

1. ¿Qué interfaz representa una tarea que puede devolver un resultado y lanzar excepciones checked, a diferencia de Runnable?

Respuesta correcta: A) Callable<V>

Callable<V> define call() que devuelve un valor V y puede lanzar excepciones checked; Runnable define run() sin retorno y sin poder lanzar checked exceptions.

2. ¿Qué provee un ExecutorService respecto a crear Thread manualmente?

Respuesta correcta: A) Un framework de alto nivel para administrar un pool de hilos reutilizables, enviar tareas (submit/execute) y gestionar su ciclo de vida

ExecutorService (java.util.concurrent) administra un conjunto de hilos, permitiendo enviar tareas sin gestionar manualmente la creación y destrucción de cada Thread.

3. ¿Qué son los 'virtual threads' introducidos en Java 21 (JEP 444)?

Respuesta correcta: A) Hilos ligeros gestionados por la JVM (no por el sistema operativo) que permiten crear millones de hilos concurrentes con bajo costo de memoria, ideales para aplicaciones con mucha I/O

Los virtual threads (Project Loom, estándar desde Java 21) son gestionados por la JVM en lugar de mapearse 1:1 a hilos del SO, permitiendo alta concurrencia con muy bajo overhead.

4. ¿Qué palabra clave permite garantizar que solo un hilo a la vez ejecute un bloque de código crítico?

Respuesta correcta: A) synchronized

'synchronized' aplicado a un método o bloque asegura exclusión mutua: solo un hilo puede ejecutar esa sección crítica sobre el mismo monitor a la vez.

5. ¿Cómo se crea rápidamente un ExecutorService basado en virtual threads en Java 21?

Respuesta correcta: A) Executors.newVirtualThreadPerTaskExecutor()

Executors.newVirtualThreadPerTaskExecutor() crea un ExecutorService que lanza una nueva virtual thread por cada tarea enviada.

6. ¿Qué método de la clase Thread permite pausar la ejecución del hilo actual durante un tiempo determinado?

Respuesta correcta: A) Thread.sleep(milisegundos)

Thread.sleep(ms) pausa la ejecución del hilo actual durante el tiempo indicado, sin liberar ningún monitor que tenga adquirido.

7. ¿Qué es una 'race condition' (condición de carrera) en programación concurrente?

Respuesta correcta: A) Una situación en la que el resultado del programa depende del orden impredecible de ejecución de múltiples hilos accediendo a datos compartidos sin la sincronización adecuada

Una race condition ocurre cuando el resultado depende del orden en que se intercalan operaciones de distintos hilos sobre recursos compartidos, produciendo resultados inconsistentes si no hay sincronización adecuada.

8. ¿Qué aporta la clase `java.util.concurrent.atomic.AtomicInteger` frente a un `int` normal en un entorno multihilo?

Respuesta correcta: A) Provee operaciones atómicas (como `incrementAndGet()`) que se ejecutan de forma indivisible sin necesidad de bloques `synchronized`, evitando condiciones de carrera

`AtomicInteger` (y clases similares) ofrece operaciones atómicas sin bloqueo explícito, útiles para contadores y acumuladores compartidos entre hilos, evitando problemas de concurrencia sin la sobrecarga de `synchronized`.

Manejo de excepciones

1. ¿Cuál es la diferencia entre una excepción 'checked' y una 'unchecked'?

Respuesta correcta: A) Las checked deben declararse (throws) o manejarse obligatoriamente en compilación (extienden `Exception`); las unchecked no obligan a manejo explícito (extienden `RuntimeException`)

Las excepciones checked (como `IOException`) obligan al compilador a exigir manejo o declaración con `throws`; las unchecked (`RuntimeException` y subclasses) no tienen esa obligación.

2. ¿Qué garantiza el bloque 'finally' en un try-catch-finally?

Respuesta correcta: A) Se ejecuta siempre, haya o no excepción, incluso si hay un return dentro del try o catch (salvo `System.exit()` o error grave de JVM)

El bloque `finally` se ejecuta siempre tras `try/catch`, independientemente de si hubo excepción o un `return`, salvo casos extremos como `System.exit()`.

3. ¿Qué ventaja ofrece 'try-with-resources' introducido en Java 7?

Respuesta correcta: A) Cierra automáticamente los recursos que implementan `AutoCloseable` al finalizar el bloque try, sin necesidad de un finally explícito

`try-with-resources` cierra automáticamente cada recurso declarado (`AutoCloseable/Closeable`) al salir del bloque, en orden inverso a su declaración, incluso si ocurre una excepción.

4. ¿Qué clase es la superclase común de `Exception` y `Error`?

Respuesta correcta: A) `Throwable`

`java.lang.Throwable` es la raíz de la jerarquía; de ella derivan `Exception` (condiciones recuperables) y `Error` (problemas graves, normalmente no manejables).

5. ¿Se puede tener múltiples bloques catch para un mismo try, capturando distintos tipos de excepción?

Respuesta correcta: A) Sí, se pueden encadenar varios catch; el orden importa: las excepciones más específicas deben ir antes que las más generales

Java permite múltiples bloques `catch`; deben ordenarse de más específico a más general, ya que colocar una excepción general (como `Exception`) antes que una más específica provoca un error de compilación por código inalcanzable.

6. ¿Qué permite el 'multi-catch' introducido en Java 7 (`catch (IOException | SQLException e)`)?

Respuesta correcta: A) Capturar varios tipos de excepción no relacionados jerárquicamente en un único bloque catch, evitando duplicar código de manejo

El `multi-catch` permite manejar varios tipos de excepción con el mismo bloque de código usando `|`, siempre que no exista relación de herencia directa entre ellas (ya que en ese caso bastaría capturar la más general).

7. ¿Qué es una excepción personalizada (custom exception) y cómo se crea típicamente?

Respuesta correcta: A) Una clase que extiende `Exception` (checked) o `RuntimeException` (unchecked) para representar un error específico del dominio de la aplicación

Se crean excepciones personalizadas extendiendo `Exception` (si debe ser checked) o `RuntimeException` (si debe ser unchecked), típicamente añadiendo constructores que reciban un mensaje y/o causa.

BigDecimal y precisión numérica

1. ¿Por qué se recomienda usar BigDecimal en lugar de double para cálculos monetarios?

Respuesta correcta: A) Porque double usa representación de punto flotante binario que introduce errores de redondeo; BigDecimal representa números decimales exactos con precisión arbitraria

double/float usan representación binaria IEEE 754 que no puede representar exactamente muchos decimales, causando errores acumulativos; BigDecimal evita esto mediante representación decimal exacta.

2. ¿Cuál es la forma recomendada de crear un BigDecimal a partir de un valor decimal exacto como 19.99?

Respuesta correcta: A) new BigDecimal("19.99") usando un String, para evitar imprecisión del double

Construir BigDecimal a partir de un double literal puede arrastrar la imprecisión binaria de ese double; usar el constructor con String es más seguro.

3. ¿Por qué comparar dos BigDecimal con equals() puede dar resultados inesperados, por ejemplo entre new BigDecimal("2.0") y new BigDecimal("2.00")?

Respuesta correcta: A) Porque equals() considera también la escala (número de decimales); para comparar solo el valor numérico se debe usar compareTo()

equals() en BigDecimal compara valor Y escala; '2.0' y '2.00' son matemáticamente iguales pero tienen distinta escala, por lo que equals() devuelve false. compareTo() sí compara solo el valor numérico.

4. ¿Qué método de BigDecimal se usa para redondear un valor a una cantidad específica de decimales con un modo de redondeo determinado?

Respuesta correcta: A) setScale(decimales, RoundingMode.HALF_UP) (u otro RoundingMode)

setScale(int newScale, RoundingMode mode) ajusta la escala (decimales) de un BigDecimal aplicando el modo de redondeo indicado, como HALF_UP, HALF_EVEN, etc.

5. ¿Por qué BigDecimal es inmutable, igual que String?

Respuesta correcta: A) Porque cada operación aritmética (add, subtract, multiply...) devuelve un nuevo objeto BigDecimal en lugar de modificar el original, garantizando consistencia y seguridad en entornos concurrentes

Al ser inmutable, cada operación como add() o multiply() retorna un nuevo BigDecimal; esto facilita el razonamiento sobre el código y la seguridad en concurrencia, similar al diseño de String.

Internacionalización (Locale, ResourceBundle)

1. ¿Qué clase representa una región geográfica, política o cultural específica para adaptar formatos y textos (i18n)?

Respuesta correcta: A) java.util.Locale

Locale encapsula idioma y país/región, usado por clases como NumberFormat, DateFormat y ResourceBundle para adaptar la salida.

2. ¿Cómo se cargan mensajes traducidos según el idioma usando ResourceBundle?

Respuesta correcta: A) ResourceBundle.getBundle("nombreBase", locale) busca automáticamente el archivo .properties más específico que coincida con el Locale dado, con fallback al bundle por defecto

ResourceBundle.getBundle busca archivos como mensajes_es_MX.properties, mensajes_es.properties, hasta mensajes.properties (default), siguiendo el algoritmo de resolución de Locale.

3. ¿Qué clase se usa para formatear números o monedas de acuerdo a un Locale específico?

Respuesta correcta: A) java.text.NumberFormat

NumberFormat.getInstance(locale) o NumberFormat.getCurrencyInstance(locale) formatean números respetando las convenciones del Locale indicado.

4. ¿Cómo se crea un Locale específico para español de México?

Respuesta correcta: A) Locale.of("es", "MX") (o el constructor equivalente new Locale("es", "MX") en versiones previas)

Se construye combinando el código de idioma ISO (es) y el código de país ISO (MX); en versiones recientes se recomienda el método de fábrica Locale.of(idioma, pais) en lugar del constructor directo.

5. ¿Qué convención de nombres de archivo sigue un ResourceBundle basado en propiedades para el Locale francés (fr_FR), con nombre base 'mensajes'?

Respuesta correcta: A) mensajes_fr_FR.properties

La convención es nombreBase_idioma_PAIS.properties, por ejemplo mensajes_fr_FR.properties, con guiones bajos separando idioma y país.

Patrones de diseño (Strategy)

1. ¿Qué problema resuelve el patrón de diseño Strategy?

Respuesta correcta: A) Permite definir una familia de algoritmos intercambiables, encapsulando cada uno en una implementación separada, de modo que el algoritmo pueda variar independientemente del código que lo usa

Strategy define una interfaz común para varios algoritmos intercambiables; el contexto delega en la estrategia concreta, permitiendo cambiarla en tiempo de ejecución sin modificar el cliente.

2. En Java moderno, ¿qué característica del lenguaje permite implementar el patrón Strategy de forma muy concisa sin crear clases separadas para cada algoritmo?

Respuesta correcta: A) Expresiones lambda que implementan una interfaz funcional que representa la estrategia

Si la interfaz Strategy es funcional, se puede pasar una lambda o method reference como estrategia en lugar de crear una clase concreta por cada algoritmo.

3. ¿Cuál es una ventaja de usar Strategy frente a múltiples bloques if-else o switch para seleccionar un algoritmo?

Respuesta correcta: A) Facilita añadir nuevas estrategias sin modificar el código existente que las usa (principio abierto/cerrado)

Strategy favorece el principio Open/Closed: se pueden añadir nuevas estrategias creando nuevas implementaciones, sin modificar el código cliente existente.

4. En el patrón Strategy, ¿qué rol cumple la clase 'Contexto'?

Respuesta correcta: A) Mantiene una referencia a una estrategia (interfaz común) y delega en ella la ejecución del algoritmo, sin conocer los detalles de la implementación concreta

El Contexto mantiene una referencia a la interfaz Strategy y delega en ella la ejecución concreta, pudiendo cambiar de estrategia en tiempo de ejecución sin modificar su propio código.

Clases anidadas e internas

1. ¿Qué es una clase interna (inner class) no estática en Java?

Respuesta correcta: A) Una clase definida dentro de otra clase que mantiene una referencia implícita a una instancia de la clase contenedora, permitiendo acceder a sus miembros de instancia

Una clase interna no estática está ligada a una instancia de la clase externa (se crea como externa.new Interna()), y puede acceder directamente a los miembros de instancia de esa clase externa.

2. ¿Cuál es la diferencia principal entre una clase anidada 'static' y una clase interna (no estática)?

Respuesta correcta: A) La clase anidada static no necesita una instancia de la clase externa para crearse y no tiene acceso implícito a los miembros de instancia de esta; la clase interna sí requiere esa instancia

Una clase estática anidada se comporta como una clase de nivel superior más, sin vínculo a una instancia externa (ClaseExterna.ClaseEstatica obj = new ClaseExterna.ClaseEstatica()); la clase interna no estática requiere una instancia externa asociada.

3. ¿Qué es una clase local (local class) en Java?

Respuesta correcta: A) Una clase definida dentro del cuerpo de un método, visible solo dentro de ese método

Una clase local se declara dentro de un bloque de método y solo es visible/utilizable dentro de ese bloque, a diferencia de las clases anónimas que no tienen nombre propio.

4. ¿Puede una clase interna no estática acceder directamente a variables private de la clase externa que la contiene?

Respuesta correcta: A) Sí, las clases internas tienen acceso completo a todos los miembros (incluidos private) de la clase externa que las contiene

Una de las principales ventajas de las clases internas es su acceso irrestricto a todos los miembros, incluidos los private, de la clase que las contiene.

API de fecha y hora (java.time)

1. ¿Qué clase del paquete java.time representa una fecha sin hora, como '2026-07-07'?

Respuesta correcta: A) LocalDate

LocalDate representa solo una fecha (año, mes, día) sin componente de hora ni zona horaria.

2. ¿Qué clase representa una fecha y hora sin información de zona horaria?

Respuesta correcta: A) LocalDateTime

LocalDateTime combina fecha y hora (LocalDate + LocalTime) sin asociar ninguna zona horaria específica. ZonedDateTime y OffsetDateTime sí incluyen esa información.

3. ¿Son mutables los objetos de las clases del paquete java.time como LocalDate y LocalDateTime?

Respuesta correcta: A) No, son inmutables; métodos como plusDays() devuelven una nueva instancia en lugar de modificar el objeto original

Al igual que String y BigDecimal, las clases del API java.time (introducido en Java 8 para reemplazar la antigua y problemática clase Date/Calendar) son inmutables; sus métodos de modificación devuelven siempre un nuevo objeto.

4. ¿Qué clase se usa para representar una cantidad de tiempo basada en fechas (años, meses, días), como la diferencia entre dos LocalDate?

Respuesta correcta: A) Period

Period representa una cantidad de tiempo en términos de años/meses/días (basado en fechas del calendario); Duration representa cantidades de tiempo basadas en segundos/nanosegundos, útil entre instantes temporales.

5. ¿Qué método se usa para calcular la diferencia entre dos LocalDate en días?

Respuesta correcta: A) ChronoUnit.DAYS.between(fecha1, fecha2)

ChronoUnit.DAYS.between(inicio, fin) calcula la cantidad de días entre dos objetos temporales; también puede usarse Period.between() para obtener años/meses/días por separado.

Autoboxing, unboxing y wrapper classes

1. ¿Qué es el 'unboxing' en Java?

Respuesta correcta: A) La conversión automática de un objeto wrapper (como Integer) a su tipo primitivo correspondiente (como int)

El unboxing convierte automáticamente un objeto wrapper (Integer, Double, etc.) a su primitivo equivalente cuando el contexto lo requiere, por ejemplo en una operación aritmética.

2. ¿Qué ocurre si se intenta hacer unboxing de un valor Integer que es null, por ejemplo al asignarlo a un int?

Respuesta correcta: A) Se lanza NullPointerException en tiempo de ejecución

Intentar convertir un Integer null a un int primitivo provoca NullPointerException en tiempo de ejecución, ya que no existe un valor primitivo que represente 'ausencia de valor'.

3. ¿Qué imprime este código?

```
Integer a = 100;  
Integer b = 100;  
System.out.println(a == b);  
Integer c = 200;  
Integer d = 200;  
System.out.println(c == d);
```

Respuesta correcta: A) true y false

Java cachea automáticamente los objetos Integer en el rango -128 a 127 (Integer Cache); por eso a==b da true (mismo objeto cacheado), pero 200 está fuera del rango cacheado, por lo que c==d compara referencias distintas y da false.

4. ¿Cuál es el método recomendado para convertir un String a un int de forma directa, sin pasar por un objeto Integer?

Respuesta correcta: A) Integer.parseInt(texto)

Integer.parseInt(String) devuelve directamente un int primitivo; Integer.valueOf(String) en cambio devuelve un objeto Integer (con posible uso de la caché).

Módulos Java (JPMS)

1. ¿Qué archivo declara un módulo en el sistema de módulos de Java (JPMS, introducido en Java 9)?

Respuesta correcta: A) module-info.java

module-info.java, ubicado en la raíz del código fuente del módulo, contiene la declaración del módulo: su nombre, qué paquetes exporta y qué módulos requiere.

2. ¿Qué directiva dentro de module-info.java indica que un módulo depende de otro módulo?

Respuesta correcta: A) requires nombreModulo;

'requires' declara una dependencia de compilación y ejecución hacia otro módulo. 'uses' se refiere a servicios consumidos, no a dependencias de módulo directas.

3. ¿Qué directiva permite que otros módulos accedan a las clases públicas de un paquete específico?

Respuesta correcta: A) exports nombrePaquete;

'exports' hace visible un paquete (sus tipos públicos) a otros módulos que lo requieran; sin exportar el paquete, sus clases permanecen encapsuladas dentro del módulo, incluso si son públicas.

4. ¿Cuál es uno de los principales beneficios del sistema de módulos (JPMS) respecto al classpath tradicional?

Respuesta correcta: A) Permite encapsulamiento fuerte a nivel de módulo: un paquete no exportado no es accesible desde fuera, incluso si sus clases son públicas

JPMS introduce encapsulamiento fuerte: solo los paquetes explícitamente exportados con 'exports' son visibles fuera del módulo, mejorando la seguridad y el diseño frente al classpath tradicional donde todo lo público era accesible.

5. ¿Qué directiva se usa en module-info.java para declarar que un módulo abre un paquete a la reflexión (por ejemplo para frameworks como Hibernate)?

Respuesta correcta: A) opens nombrePaquete;

'opens' permite el acceso reflexivo (incluso a miembros privados vía reflection) a un paquete desde otros módulos, útil para frameworks que instancian objetos dinámicamente.